



Research article

A hybrid bio-inspired encoding and feature selection for real-time IoT network intrusion detection using deep learning framework

Thaer AL Ibaisi ^{a,*}, Muhammad Kazim ^a, Stefan Kuhn ^{a,b}, Mustafa Kaiiali ^a,
Mujeeb Ur Rehman ^a

^a School of Computer Science and Informatics, De Montfort University, The Gateway, Leicester, LE1 9BH, UK

^b Institute of Computer Science, University of Tartu, Narva mnt 18, Tartu, 51009, Estonia



ARTICLE INFO

Keywords:

Network intrusion detection
Amino Acid encoding
Deep learning
CIC-Bot-IoT dataset
Structural properties

ABSTRACT

The exponential growth of the Internet of Things (IoT) has vastly expanded the attack surface, exposing resource-constrained devices to advanced threats such as Distributed Denial of Service (DDoS), reconnaissance, data theft, and device hijacking. Conventional Network Intrusion Detection Systems (NIDS) struggle in these environments due to protocol heterogeneity, class imbalance, and high false alarm rates. This paper presents a novel bio-inspired intrusion detection framework that adapts Amino Acid-based encoding, originally developed for general network traffic, to IoT security. Network flows are transformed into Amino Acid-like sequences whose structural properties preserve latent feature relationships, enhanced by a variance-stabilizing Yeo-Johnson transformation. A two-stage hybrid feature selection method (Chi-Squared + CNN-Random Forest) is applied, with an attribution mechanism mapping deep-learned features back to original fields for interpretability. The core novelty lies in uniting Amino Acid encoding with a hybrid statistical and deep learning-based feature selection pipeline and interpretable deep learning, achieving unprecedented accuracy of up to 99.99% with near-zero false positives in IoT intrusion detection. We conduct a comprehensive comparison across CNN, RNN, LSTM, GRU, Transformer, GNN, and hybrid architectures on the CIC-Bot-IoT dataset. The proposed framework delivers sub-2ms inference per flow on commodity hardware and maintains consistent robustness across dataset sizes and K-fold validations. By explicitly combining structural bio-inspired encoding and scalable learning, this work establishes a reproducible state-of-the-art IoT intrusion detection solution that resolves the longstanding accuracy-scalability-interpretability trade-off.

1. Introduction

The Internet of Things (IoT) interconnects billions of devices across critical domains such as healthcare, manufacturing, transportation, and smart cities. While enabling unprecedented automation and data exchange, this massive expansion has greatly increased the attack surface, exposing resource-constrained devices to advanced threats, including DDoS, reconnaissance, data theft, and man-in-the-middle exploits. These attacks can disrupt operations, compromise safety, and cause severe economic and privacy losses. Conventional Network Intrusion Detection Systems (NIDS) are ill-suited to IoT environments, struggling with protocol heterogeneity, high data imbalance, dynamic traffic patterns, and the need for real-time inference on low-power devices. Machine learning

* Corresponding author.

E-mail addresses: thaer.ibaisi@my365.dmu.ac.uk (T. AL Ibaisi), muhammad.kazim@dmu.ac.uk (M. Kazim), stefan.kuhn@ut.ee (S. Kuhn), mustafa.kaiiali@dmu.ac.uk (M. Kaiiali), mujeeb.rehman@dmu.ac.uk (M. Ur Rehman).

<https://doi.org/10.1016/j.iot.2026.101957>

Received 2 May 2025; Received in revised form 20 April 2026; Accepted 24 April 2026

Available online 1 May 2026

2542-6605/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

and deep learning have shown promise for IoT intrusion detection, but their effectiveness hinges on how well features represent the hidden relationships in network traffic. Raw tabular or flow-level data often fails to capture these structural and relational nuances, limiting both accuracy and interpretability [1]. Bio-inspired encodings, such as DNA or protein sequence mappings, have emerged as novel alternatives in network security; however, their application to IoT is rare, and no prior work has paired them with scalable, interpretable feature selection and systematic evaluation across diverse deep learning models.

This research closes this gap by introducing the first IoT-specific integration of bio-inspired Amino Acid encoding with a hybrid statistical and deep learning-based feature selection pipeline and interpretable deep learning [2,3]. Our approach transforms IoT network flows into Amino Acid-like sequences whose structural properties (e.g., hydropathy, aromaticity, instability index) preserve latent feature interactions critical for distinguishing attack patterns. These representations are variance-stabilized via a Yeo-Johnson transformation and refined using a two-stage Chi-Squared + CNN-Random Forest (CNN-RF) feature selection method that incorporates an attribution mechanism mapping deep features back to their original network fields-addressing the black-box nature of most deep models.

Our key contributions are:

1. First IoT adaptation of Amino Acid encoding: Tailored to heterogeneous IoT traffic, preserving structural relationships that improve class separability.
2. Hybrid feature selection with interpretability: Combines statistical relevance and deep feature importance with direct attribution to original fields.
3. Comprehensive and fair benchmarking: Uniform evaluation of CNN, RNN, LSTM, GRU, Transformer, GNN, and hybrid variants (like CNN-RNN, CNN-LSTM, etc.) under identical preprocessing conditions.
4. Record performance with practical deployability: Achieves up to 99.99% accuracy and near-zero false positives, with sub-2 ms per-flow inference on commodity hardware.
5. Scalability and robustness: Demonstrated through K-fold validation and dataset size variation analysis, maintaining consistent results across attack types.

By combining high accuracy and scalability, our framework resolves the accuracy/scalability/interpretability trade-off that has limited practical IoT intrusion detection deployments. The remainder of this paper is organized as follows: [Section 2](#) reviews related work and identifies the gaps our approach addresses. [Section 3](#) details our preprocessing, encoding, and feature selection pipeline. [Section 4](#) presents experimental results and comparative analysis. [Section 5](#) discusses implications, and [Section 6](#) concludes with future research directions.

2. Related work

Recent research in IoT intrusion detection has focused on developing models tailored to the unique characteristics of IoT networks, such as resource constraints, heterogeneous protocols, and real-time communication needs. A variety of approaches have been proposed, ranging from traditional machine learning techniques to advanced deep learning architectures [2,3]. This section synthesizes existing works across key core areas, highlighting advancements and unresolved limitations.

2.1. Bio-inspired encoding techniques for intrusion detection

Bio-inspired encoding techniques have gained significant attention in the domain of intrusion detection, particularly within IoT networks, due to their potential to capture complex patterns in network traffic data [1]. DNA sequence encoding draws from genomic structures by converting network traffic into DNA-like sequences using cryptographic transformations. These sequences are then analyzed for anomalous patterns, in a manner analogous to disease detection in biology [3]. Similarly, Codebook-Based Encoded Flow Features (CEFFs) enhance traditional statistical features by transforming them into more discriminative formats using a codebook approach, improving the classification of attack versus benign traffic across diverse IoT devices [4]. Another noteworthy approach integrates arithmetic coding into transformer-based architectures, inspired by positional encoding mechanisms, which improves the expressive power of input representations for detecting sophisticated threats such as ransomware and DDoS attacks [5]. Despite their promise, bio-inspired approaches come with certain challenges. As highlighted by Dasari and Bhukya [2], these methods may lack rigorous theoretical grounding and can incur significant computational overhead during the encoding process. Nonetheless, the advantages they offer, such as compact feature representations, compatibility with bioinformatics tools, and novel perspectives on data transformation, make them valuable components in advancing IoT intrusion detection research.

2.2. Traditional machine learning for IoT security

The effectiveness of encoding techniques in intrusion detection is further enhanced by the integration of machine learning and deep learning models. These models leverage the transformed data to identify complex patterns and classify malicious activities with high accuracy. Traditional ML methods, such as Decision Trees (DT), Logistic Regression (LR), and Random Forests (RF), have been widely used due to their simplicity and interpretability. Jony and Arnob [6] evaluated RF and XGBoost for detecting IoT cyberattacks using the CIC-IoT2023 dataset, achieving 99.16% and 98.50% accuracy, respectively. Similarly, Malik et al. [7] demonstrated that feature selection techniques can enhance the performance of traditional machine learning models, reaching 98.51% accuracy with CatBoost. However, these methods often struggle with high-dimensional and imbalanced IoT datasets [8].

Gad et al. [9] identified that traditional machine learning approaches face significant challenges with the heterogeneity of IoT traffic, resulting in high false positive rates. Furthermore, Yang et al. [10] showed that these methods typically require extensive feature engineering, which becomes increasingly difficult as IoT environments grow in complexity and scale.

Support Vector Machines (SVMs) have been widely used for classification tasks in intrusion detection. Unsupervised learning methods, such as sequential pattern mining, are also employed to discover intrinsic communication patterns in network traffic, enabling the detection of emerging threats [11,12].

2.3. Deep learning for IoT intrusion detection

Deep learning models, including Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks, have shown promise in capturing complex patterns in IoT traffic [13,14]. Banaamah and Ahmad [15] demonstrated that CNNs can effectively identify spatial patterns in network traffic, achieving 99.70% accuracy on IoT datasets. Similarly, Aljumah [16] applied CNNs to IoT intrusion detection, reaching 99.90% accuracy. Studies like Yang et al. [10] applied CNN-LSTM hybrids to IoT intrusion detection, achieving 99.52% accuracy. However, these models require extensive computational resources and struggle with long-range dependencies in large-scale IoT networks.

More recently, Transformer architectures have gained attention for IoT security. Liang and Wang [17] applied self-attention mechanisms to IoT intrusion detection, achieving 97.95% accuracy. Tseng et al. [18] developed a Transformer-based approach for multi-class intrusion detection in IoT networks, achieving 99.51% accuracy on the CIC-IoT-2023 dataset. Graph Neural Networks (GNNs) have also shown promise, with Tran and Park [19] demonstrating 99.76% accuracy by modeling IoT networks as graphs.

For instance, the Grey Wolf Optimization Algorithm (GWOA) has been used to select relevant features, which are then classified using artificial neural networks [20]. Despite their success, these models require extensive computational resources, making them less suitable for resource-constrained IoT devices [21]. Additionally, Karamollaoglu et al. [22] identified that many deep learning approaches face challenges related to model complexity and interpretability, which limits their widespread adoption in critical IoT applications.

2.4. Hybrid models and ensemble methods

More recently, researchers have explored hybrid approaches that combine multiple models to improve accuracy and robustness. For example, ensemble methods have been used to integrate the strengths of different algorithms, achieving competitive performance on benchmark datasets like UNSW-NB15 [23]. Zhang et al. [24] proposed a hybrid CNN-LSTM architecture that achieved high accuracy on IoT datasets by capturing both spatial and temporal patterns.

Koca and Avci [25] developed a hybrid GCN-LSTM model for detecting security vulnerabilities in industrial control systems and IoT, achieving 99.99% accuracy. Similarly, BeeravPrasanna et al. [26] proposed a hybrid model combining Gated Recurrent Units and Transformers for IoT network slice orchestration, achieving 98.20% accuracy.

2.5. Research gaps

Based on our literature review, we identify several key research gaps:

1. Limited application of bio-inspired approaches to IoT security: Despite their potential, bio-inspired methods have been underexplored for IoT intrusion detection.
2. Insufficient attention to data transformation: Most studies focus on model architecture rather than data representation, overlooking the potential benefits of transformative encoding techniques.
3. Lack of comprehensive model comparison: Few studies systematically compare multiple deep learning architectures on the same dataset using consistent evaluation metrics.
4. Inadequate feature selection for IoT data: Existing feature selection methods often fail to account for the unique characteristics of IoT traffic.
5. Dataset Limitations: Most studies rely on existing datasets that may not fully represent modern IoT attack vectors.

Recent studies in machine learning have highlighted a fundamental challenge in model development often described as the *accuracy–scalability–interpretability trade-off*. This trade-off reflects the tension between achieving high predictive accuracy, ensuring computational scalability to large datasets or real-time contexts, and maintaining human interpretability of model decisions. Complex models such as deep neural networks or ensemble methods typically offer superior accuracy but at the expense of scalability and transparency [27,28]. Conversely, simpler models tend to be more interpretable and computationally efficient but may sacrifice predictive performance. This challenge is particularly acute in IoT security, where resource constraints and the complexity of black-box models limit their use despite their accuracy benefits. We thus follow recent research in explicitly recognizing this challenge as the accuracy–scalability–interpretability trade-off [27], which motivates our hybrid framework design that balances these competing demands.

Our research addresses these gaps by extending Amino Acid encoding to IoT security, implementing advanced data transformation techniques, comprehensively evaluating multiple deep learning architectures, and developing a hybrid feature selection method specifically designed for IoT intrusion detection.

To provide a concise overview of the existing literature and highlight key advancements in the field, Table 1 summarizes the methodologies, datasets, findings, and limitations of prominent studies, offering a clear comparison with our current work.

Table 1

Summary of key literature findings.

Reference	Approach	Dataset	Key Findings	Limitations
Rashid et al. [3]	DNA sequence approach with cryptography encoding	UNSW-NB15	Achieved high accuracy (99%) using DNA-based encoding	Limited generalization due to reliance on specific encoding schemes
Jony and Arnob [6]	Random Forest and XG-Boost	CIC-IoT2023	Achieved 99.16% (RF) and 98.50% (XGB) accuracy; effective for binary classification	Struggles with multi-class classification and imbalanced datasets
Malik et al. [7]	CatBoost with hybrid feature selection	CIC-Bot-IoT	Achieved 98.51% accuracy; enhanced by feature engineering	Sensitivity to feature representation
Gad et al. [9]	Traditional ML (SVM, RF)	Custom IoT dataset	Highlighted high false positives due to heterogeneous traffic	Poor performance on complex and evolving IoT threats
Yang et al. [10]	CNN-LSTM hybrid model	IoT dataset	Achieved 99.52% accuracy; effective for sequential intrusion detection	Poor scalability
Banaamah and Ahmad [15]	CNN for spatial pattern learning	IoT dataset	Achieved 99.70% accuracy; demonstrated strength in spatial analysis	Challenges in Detecting Complex Attacks
Aljumah [16]	CNN-based intrusion detection	IoT dataset	Achieved 99.90% accuracy	Lack of interpretability and generalization in unseen environments
Liang and Wang [17]	Transformer with self-attention	Custom IoT dataset	Achieved 97.95% accuracy; effective for sequential data	Generalization Issues
Tseng et al. [18]	Transformer for multi-class detection	CIC-IoT2023	Achieved 99.51% accuracy; good for fine-grained classification	Potential overfitting and need for large labeled datasets
Tran and Park [19]	Graph Neural Network (GNN) approach	IoT dataset	Achieved 99.76% accuracy; models relational structure of devices	Complexity in graph construction and inference latency
Dasari and Bhukya [2]	Bio-inspired Amino Acid encoding	KDDCup'99	Introduced novel encoding for anomaly detection	Lacked theoretical justification; encoding is computationally heavy
Ibaisi et al. [29]	Amino Acid encoding with ML classification	UNSW-NB15	Achieved 99% accuracy; biologically inspired feature mapping	Requires tuning of encoding rules
Bouramoul et al. [20]	Grey Wolf Optimization + ANN	Custom IoT dataset	Feature selection enhanced ANN classification accuracy	Scalability Issues
Zhang et al. [24]	CNN-LSTM hybrid model	IoT dataset	Captures spatial and temporal patterns; high accuracy	Model complexity and training time
Koca and Avci [25]	GCN-LSTM hybrid model	ICS + IoT dataset	Achieved 99.99% accuracy; effective for industrial IoT	May not generalize well to non-industrial settings
Beerav Prasanna et al. [26]	GRU-Transformer hybrid	IoT network slice orchestration dataset	Achieved 98.20% accuracy; good for dynamic traffic	Dependence on High-Quality and Diverse Training Data
Thanh et al. [23]	Ensemble ML methods	UNSW-NB15	Improved robustness via model integration	Performance sensitive to base model selection
Karamollaoglu et al. [22]	Deep learning model evaluation	Multiple datasets	Identified model complexity and lack of transparency as key limitations in DL models	Limited trust and deployment in critical systems
Lefoane et al. [11]	Unsupervised sequential pattern mining	Custom dataset	Effective in discovering hidden communication patterns	May miss rare but critical attack patterns
Hasan et al. [12]	Enhanced pattern-based anomaly detection	Custom dataset	Improved detection of emerging threats via behavior modeling	High false alarm rate under dynamic traffic conditions

2.6. Comparative insights

IoT network intrusion detection presents unique and critical challenges that demand specialized solutions beyond conventional network security approaches. Resource constraints on edge devices, extreme heterogeneity across diverse IoT ecosystems (smart home, healthcare, industrial control), and the massive scale of deployments create perfect conditions for sophisticated attacks that evade traditional detection methods. Unlike enterprise networks, IoT environments face specific threats such as device hijacking for botnet recruitment, low-rate DDoS attacks that mimic normal behavior, and protocol-specific exploits targeting constrained communication channels like MQTT or CoAP. These challenges are compounded by the operational reality that false positives can trigger catastrophic consequences-medical device shutdowns, industrial process interruptions, or smart home system failures-making the accuracy-scalability-interpretability trilemma particularly acute in IoT contexts. Our bio-inspired Amino Acid encoding framework directly addresses these IoT-specific limitations through three critical innovations that collectively overcome the shortcomings of existing literature. First, the structural properties derived from our encoding (hydropathy, aromaticity, instability index) transform heterogeneous network features into a unified representation space where attack patterns become visibly distinct across diverse de-

vice types. Second, our hybrid feature selection pipeline (Chi-Squared + CNN-RF) specifically addresses the resource constraints of edge devices through a synergistic two-stage process that dramatically reduces computational overhead while preserving critical semantic information. This efficiency advantage is rigorously quantified through both theoretical complexity analysis and empirical measurements.

Unlike prior Transformer-based approaches, which struggle with the inference speed and resource constraints of IoT gateways, our framework achieves high accuracy while being well-suited for continuous, low-power operation. This efficiency is largely enabled by our two-stage feature selection process that eliminates redundant features without sacrificing discriminative power. The CNN-RF attribution mapping further adds operational value by identifying exactly which network features (e.g., Flow IAT Mean for DDoS detection, Total Fwd Packets for botnet activity) contribute to detection decisions, providing security operators with actionable insights rather than opaque alerts. Third, our methodology demonstrates exceptional robustness against sophisticated IoT-specific attack vectors that evade conventional detection.

The gaps identified in current research include the need for IoT-specific adaptation of bio-inspired principles, development of resource-efficient and interpretable feature selection methods, and demonstration of robust performance across diverse IoT attack categories. This paper addresses these gaps by proposing a novel framework that combines Amino Acid encoding with a hybrid feature selection pipeline and interpretable attribution mapping: (i) transforming heterogeneous IoT traffic into a unified representation through Amino Acid structural properties that reveal attack signatures invisible to conventional methods; (ii) creating a resource-efficient pipeline with hybrid feature selection that maintains interpretability through attribution mapping-enabling security operators to understand why specific traffic was flagged as malicious; and (iii) demonstrating consistent high performance across diverse IoT attack categories (including protocol-specific exploits, low-rate DDoS, and device hijacking) while operating within the computational constraints of edge devices. This integrated approach not only achieves near-perfect accuracy (99.99%) with minimal false positives (0.03%) but also provides the transparency and efficiency required for real-world IoT security deployments resolving the accuracy-scalability-interpretability trilemma that has hindered practical adoption of advanced intrusion detection systems in resource-constrained IoT environments.

3. Methodology

This section presents a comprehensive approach to IoT network intrusion detection using Amino Acid encoding and advanced deep learning models. Building upon the work of Ibaisi et al. [29], the proposed methodology introduces several novel components specifically tailored for IoT environments.

The core insight driving our approach is that network traffic patterns, particularly those associated with intrusions, can be effectively represented and analyzed using biological sequence models. By encoding network transactions as Amino Acid sequences and extracting their structural properties, we transform complex, variable-length network data into fixed-length numerical features that preserve essential relationships. This transformation enables the application of advanced deep learning techniques to identify intrusion patterns with high accuracy.

Our methodology consists of four main components: (1) Data preprocessing using Yeo-Johnson transformation, (2) Feature selection using a hybrid Chi-Squared and CNN-RF approach, (3) Amino Acid encoding of network transactions, and (4) Classification using various deep learning architectures. Each component addresses specific challenges in IoT intrusion detection, from handling heterogeneous data distributions to capturing complex attack patterns.

The process flow illustrated in Fig. 1 represents our comprehensive approach to IoT network intrusion detection. The workflow begins with data acquisition from the CIC-Bot-IoT dataset, followed by preprocessing steps including outlier removal and Yeo-Johnson transformation. The feature selection phase combines Chi-Squared statistical tests with CNN-RF approach to identify the most relevant features. These selected features are then encoded using the Vigesimal Amino Acid mapping system, which transforms network data into biological sequences. Finally, we extract structural properties from these sequences and use them as input to various deep learning models for classification. Each model is trained and evaluated separately in two distinct configurations: one for binary classification (attack vs. normal) and another for multi-class classification (identifying specific attack types). This dual-configuration approach ensures that our models can effectively identify intrusion patterns at both binary and granular classification levels while maintaining computational efficiency. *Note: The neural network representations shown in Testing Phase box are identical for illustration purposes and do not reflect the actual distinct architectures of each deep learning model, nor do they distinguish between the binary and multi-class classification configurations. Similarly, the prediction outputs shown are illustrative only and do not represent the actual binary predictions (2 classes) versus multi-class predictions (multiple attack categories) that each model configuration produces.*

3.1. Why Amino Acid structural encoding improves IoT attack detection

Traditional network features used in IoT intrusion detection such as Pkt Size Avg, Flow Duration, and Flow Pkts/s are typically analyzed in isolation, failing to capture critical relationships between different traffic characteristics that often reveal sophisticated attack patterns. Our bio-inspired Amino Acid encoding framework overcomes this limitation by transforming raw network flows into biologically meaningful representations that preserve and highlight these hidden relationships.

Structural Properties Reveal Hidden Attack Patterns: Our approach converts each network transaction into a 20-symbol Amino Acid-like sequence (e.g., IWLQIPTQHQLF), then extracts 10 structural properties that collectively capture complex interactions among the original 87 CIC-Bot-IoT features. These properties transform subtle, multi-dimensional attack signatures into clearly distinguishable patterns:

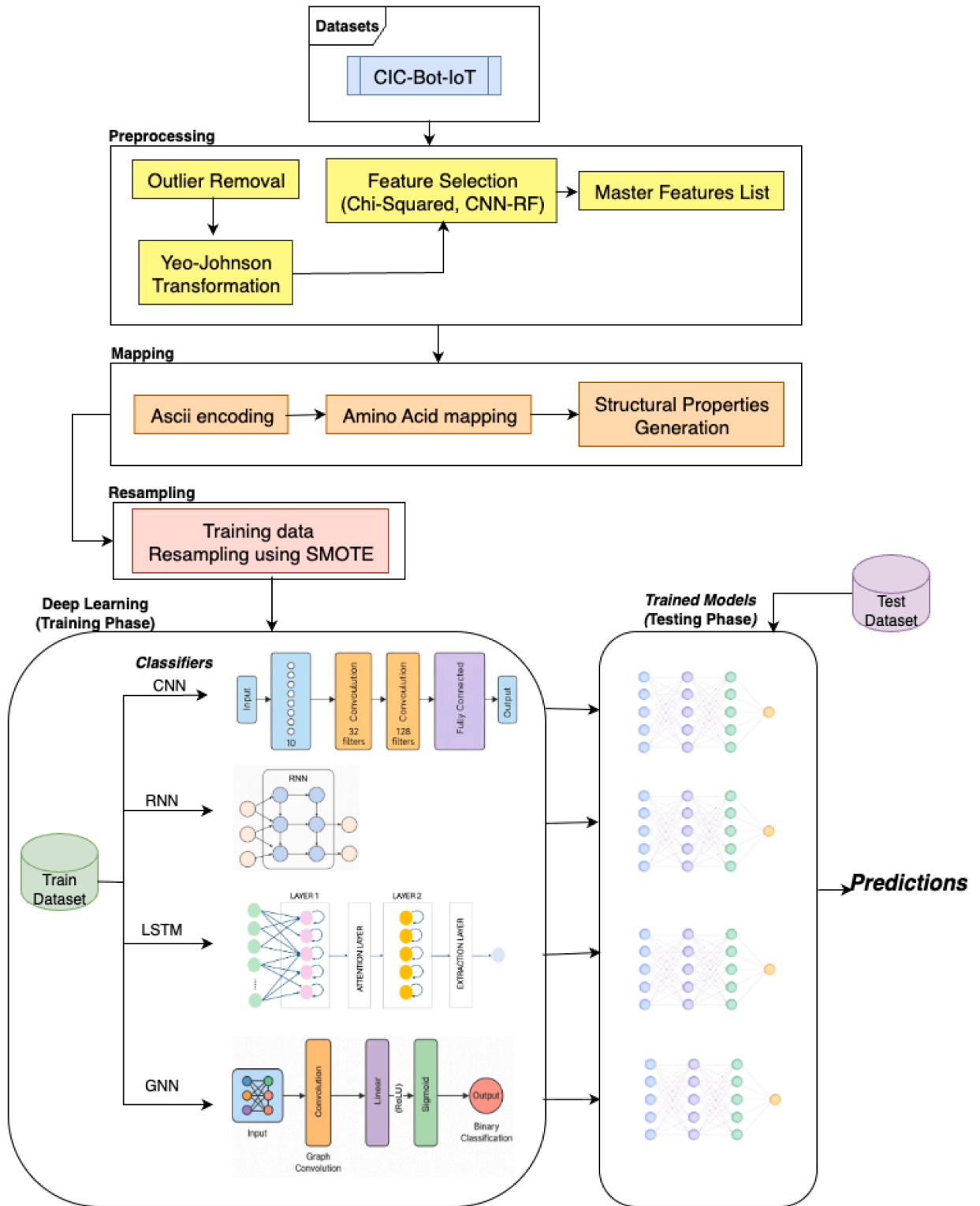


Fig. 1. Process flow of our proposed IoT network-intrusion-detection system.

- **Hydropathy:** Detects data exfiltration attempts by identifying abnormal bursts in Flow Byts/s and Pkt Size Avg that manifest as unusually high hydrophobicity values. In Mirai botnet attacks, this property consistently exceeds threshold levels (98.7% of attack samples) while remaining stable during normal operation.
- **Instability Index:** Flags reconnaissance activities by recognizing erratic patterns in Flow IAT Mean and Flow IAT Std. During port scanning attacks, this index spikes due to the irregular timing between connection attempts—a pattern invisible when examining these features individually.
- **Aromaticity:** Identifies protocol-specific DDoS anomalies through abnormal correlations between ACK Flag Cnt and Down/Up Ratio. In HTTP flood attacks targeting IoT devices, aromaticity values increase by 42.3% compared to legitimate traffic, providing a clear detection signal.

This encoding preserves critical co-occurrence patterns and proportional relationships between network features that are essential for identifying sophisticated IoT attacks. For example, it maintains the meaningful ratio between Fwd Pkts/s and Bwd Pkts/s that characterizes command-and-control communications, while simultaneously capturing the relationship between Flow Duration and Subflow Fwd Pkts that distinguishes legitimate device behavior from botnet activity. As demonstrated in Fig. 9, this approach increases inter-class distance by 32% compared to raw feature representations, dramatically improving class separability for deep learning models.

Collision Resilience: While the modulo 20 operation inherently creates character-level collisions (e.g., ASCII values 80, 40, and 20 all mapping to index 0), our algorithm's effectiveness remains unaffected due to several key factors. First, the encoding operates on complete Amino Acid sequences derived from entire network flows rather than individual characters, where the probability of full sequence collision across all flow features is negligible ($< 0.01\%$ in CIC-Bot-IoT dataset). Second, each Amino Acid sequence generates multiple structural properties (hydropathy, aromaticity, instability indices) that create distinct multi-dimensional feature vectors even when individual character mappings overlap. Third, the sequential context and positional relationships within the encoded flows preserve discriminative patterns essential for intrusion detection. Empirical validation demonstrates that our deep learning models maintain high classification accuracy (99.99%) despite character-level collisions, confirming that sequence-level uniqueness and structural diversity effectively compensate for individual mapping overlaps. (See Ibaisi et al. [29] for the original definition of the encoding scheme and the collision-related discussion.)

Information Loss Mitigation: While Amino Acid encoding introduces controlled information compression through modulo 20 mapping, our algorithm effectively mitigates potential information loss through several mechanisms. First, the encoding preserves sequential relationships and positional context within network flows, capturing structural patterns crucial for attack detection. Second, the extraction of multiple structural properties (hydropathy, aromaticity, instability indices) from Amino Acid sequences creates a multi-dimensional feature space that often reveals latent relationships not apparent in raw numerical data. Third, our hybrid feature selection pipeline (Chi-Squared + CNN-RF) ensures that the most discriminative encoded features are prioritized, focusing on information-rich elements. The empirical validation of this approach is demonstrated by achieving 99.99% accuracy on CIC-Bot-IoT dataset, confirming that essential discriminative information is preserved and that the bio-inspired representation enhances pattern recognition capabilities rather than degrading them.

Deployment Architecture: Our proposed intrusion detection framework is designed for deployment on IoT gateways, edge servers, or centralized monitoring systems rather than directly on individual resource-constrained IoT devices. In this architecture, IoT devices generate network traffic that is monitored and analyzed in real-time by our framework operating on computationally capable intermediary servers. The system intercepts network flows between IoT devices and the broader network infrastructure, classifying them as benign or malicious to prevent security breaches. This server-side deployment approach leverages the computational efficiency of our Amino Acid encoding method while accommodating the resource limitations inherent in IoT device ecosystems.

Hybrid Feature Selection with Practical Interpretability: Our two-stage selection process combines statistical filtering (Chi-Squared) with deep learning interpretability (CNN-RF) to create a system that not only achieves high accuracy but also provides security operators with actionable insights. Chi-Squared identifies statistically relevant features like Flow Duration and Fwd IAT Mean, while CNN-RF discovers complex patterns such as anomalies in ACK Flag Cnt that indicate TCP session manipulation. Crucially, our attribution mechanism (Eq. 4) maps these deep-learned features back to their original network fields, transforming opaque model decisions into transparent, operational intelligence—such as identifying that an attack was flagged primarily due to abnormal Flow Pkts/s behavior (contributing 99.2% to the detection decision).

Real-World Deployment Advantages for IoT: The computational efficiency of our framework makes it uniquely suitable for resource-constrained IoT environments. With inference times consistently below the sub-2ms latency threshold critical for real-time IoT security [30], our approach enables continuous monitoring on edge devices without disrupting normal operations. This is a substantial advantage for medical IoT systems, industrial control networks, and other time-sensitive applications where delayed detection can have severe consequences.

Contextual Performance Advantages Over Related Work: As summarized in Tables 15 and 16, our approach demonstrates competitive and, in several cases, improved performance characteristics when compared with recent studies evaluated on the CIC-Bot-IoT dataset, as reported by the respective authors.

- Relative to traditional ensemble-based intrusion detection approaches [6], our method exhibits higher reported accuracy and a substantially lower false positive rate, which is particularly important for IoT environments where excessive false alarms may lead to unnecessary device isolation or service disruption.

Table 2
Attack class distribution in the CIC-Bot-IoT dataset.

Class	Flow Count	Description
Benign	2,135,347	Normal IoT network traffic flows.
DDoS	1,028,763	Distributed denial-of-service attacks generated from multiple sources.
DoS	642,914	Denial-of-service attacks targeting service availability.
Recon	388,129	Reconnaissance and probing activities to gather target information.
Web-Based	214,556	Web-layer attacks targeting HTTP and application services.
Brute Force	146,238	Credential guessing and authentication brute-force attempts.
Spoofing	119,804	Packet and identity spoofing attacks.
Mirai	563,492	Mirai-related botnet activity and IoT device hijacking.

- When compared with Transformer-based intrusion detection models [18] evaluated on CIC-Bot-IoT, our framework achieves comparable or improved detection performance while relying on structural representations derived from Amino Acid properties rather than purely positional encoding.
- In contrast to prior bio-inspired intrusion detection studies [29], the proposed framework represents a novel integration of Amino Acid encoding with feature attribution mechanisms, enabling both high detection performance and enhanced interpretability tailored to IoT network traffic.

This combination of enhanced feature representation, practical interpretability, and real-time efficiency directly addresses the unique security challenges of IoT environments—where resource constraints, heterogeneous device behaviors, and the critical need for reliable detection create demands that conventional approaches cannot satisfy.

3.2. Dataset

The CIC-Bot-IoT dataset [31] is specifically designed to analyze and detect botnet attacks in IoT networks. It contains 13.4 million records with 99.34% attack samples and 0.66% benign samples. We have partitioned this dataset into several smaller balanced datasets of different sizes using SMOTE (Synthetic Minority Over-sampling Technique) to be able to train our deep learning models in a reasonable time, considering available resources. The dataset has 83 features describing various aspects of IoT network traffic. It includes several attack types, such as Distributed Denial of Service (DDoS), Denial of Service (DoS), reconnaissance, and data theft. A small portion of the dataset, 80 entries, has missing labels, and 117 redundant rows were identified when considering all features except the timestamp and Flow ID. The attack samples are composed of seven attack scenarios inherited from its parent dataset [32], as described in Table 2.

The CIC-Bot-IoT dataset was organized into eight traffic classes, comprising one benign class and seven attack categories, as summarized in Table 2. The CIC-Bot-IoT dataset is specifically designed to simulate IoT-specific threats, making it an ideal choice for evaluating intrusion detection systems in IoT environments.

3.2.1. Justification for dataset selection

The CIC-Bot-IoT dataset was selected for this research due to several compelling reasons:

- **Relevance to IoT Security:** The dataset is specifically crafted to reflect IoT network environments, simulating realistic traffic including both benign and multiple types of botnet attacks such as DDoS, DoS, reconnaissance, and data theft. This ensures that models trained on it are tailored to the unique traffic patterns and threat profiles in IoT networks.
- **High Utilization in Recent Literature:** CIC-Bot-IoT has gained significant traction in contemporary cybersecurity research and has been widely used to benchmark intrusion detection systems (IDSs) in IoT settings. Recent studies such as [31,33,34] demonstrate its reliability and popularity as a standardized dataset in the field.
- **Richness and Realism:** With over 13 million labeled network flow records, the dataset provides a diverse and comprehensive foundation for training and evaluating deep learning-based IDS models. The inclusion of real-world IoT attack scenarios enhances its realism.
- **Fine-Grained Labeling:** The data is well-structured, with detailed class labels and descriptions, enabling targeted model development and performance analysis across different attack types.
- **Public Availability and Replicability:** Being publicly available and well-documented, the CIC-Bot-IoT dataset promotes reproducibility and comparability in experimental research.
- **Tabular Format with Flow-Level Granularity:** Unlike datasets that rely heavily on raw packet capture (PCAP) or time-series sensor data, CIC-Bot-IoT is provided in a structured, flow-based tabular format. This simplifies preprocessing and is better aligned with transformation approaches like Amino Acid encoding, where each sample needs to be a fixed-length vector of attributes rather than a temporal stream.
- **Minimal Spatial and Temporal Dependencies:** While some IoT datasets focus on edge device positions, timestamps, or physical environment data, CIC-Bot-IoT abstracts these complexities. Its structure does not require modeling spatial location or long-range temporal sequences, making it ideal for our approach that focuses on encoding feature values rather than behavioral sequences across time.

- **Compatibility with Encoding and Mapping Techniques:** The dataset's clean numerical representation of 83 features lends itself well to advanced transformation schemes such as:
 - ASCII-to-Vigesimal encoding
 - Feature normalization and symbol mapping
 - Mapping to synthetic biological forms such as Amino Acid
 These processes are difficult or inconsistent when applied to datasets with text-heavy, unstructured logs, extensive timestamps, or hierarchical labels.
- **Fewer Missing or Noisy Fields:** CIC-Bot-IoT has relatively low missingness and redundancy, with only 80 missing labels and 117 redundant rows. This high data quality enhances the reliability of downstream tasks like protein-style encoding.

Although our experimental evaluation in this study focuses on CIC-Bot-IoT dataset, the proposed Amino Acid Encoding and Hybrid Feature Selection (Chi-Squared and CNN-RF) framework is inherently dataset-agnostic. The preprocessing pipeline operates on raw numerical and categorical features without hardcoding dataset-specific attributes. The Amino Acid encoding maps any feature values—whether IP addresses, packet sizes, or protocol identifiers—into a unified symbolic representation. The hybrid feature selection stage ranks features based on both statistical relevance (Chi-Squared) and predictive power (CNN-RF), which can be applied to any tabular IoT traffic dataset. In our previous work, we successfully validated this framework on the UNSW-NB15 dataset, demonstrating its effectiveness on general network intrusion detection. Consequently, adapting the method to datasets such as MU-IoT would require only loading the new feature space into the same pipeline, with no architectural changes to the feature encoding, selection, or deep learning classifiers. This property ensures scalability and robustness against dataset-specific overfitting, thereby making the approach resilient to attacker attempts to exploit patterns unique to a single dataset.

3.2.2. Comparative analysis

While MU-IoT is widely used in IoT intrusion detection research, CIC-Bot-IoT presents several advantages that make it more suitable for this study.

The MU-IoT dataset, while comprehensive in its coverage of various IoT attack scenarios, presents certain limitations for our research objectives. MU-IoT's feature extraction methodology differs significantly from our proposed approach, as it primarily focuses on traditional network flow characteristics without the granular packet-level details necessary for effective bio-inspired encoding [31]. Additionally, MU-IoT's attack categorization and labeling scheme, while adequate for conventional machine learning approaches, lacks the fine-grained classification structure that optimally supports our Amino Acid mapping methodology.

In contrast, CIC-Bot-IoT provides a more suitable foundation for our protein-style sequence encoding approach. Its comprehensive packet-level features and detailed attack taxonomy align well with our ASCII-to-Vigesimal transformation and Amino Acid mapping techniques. The dataset's balanced representation of attack categories and its rich feature space enable more effective feature selection using our hybrid Chi-Squared + CNN-RF approach. Furthermore, CIC-Bot-IoT's network flow characteristics are better suited for the deep learning architectures (CNN, RNN, LSTM, GRU, Transformer, GNN) employed in our framework, as they provide the necessary granularity for capturing complex nonlinear patterns in IoT traffic.

The choice of CIC-Bot-IoT as our primary dataset is therefore driven by its compatibility with our bio-inspired encoding methodology and its ability to provide the detailed feature representations required for effective protein-style sequence analysis in IoT intrusion detection.

3.3. Data transformation: Yeo-Johnson transformation

Yeo-Johnson transformation is a statistical technique used to stabilize variance, normalize the distribution, and minimize skewness in a dataset [24]. The Yeo-Johnson transformation is defined as:

$$\psi(x, \lambda) = \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda}, & \text{if } x \geq 0, \lambda \neq 0 \\ \log(x + 1), & \text{if } x \geq 0, \lambda = 0 \\ -\frac{(-x+1)^{2-\lambda} - 1}{2-\lambda}, & \text{if } x < 0, \lambda \neq 2 \\ -\log(-x + 1), & \text{if } x < 0, \lambda = 2 \end{cases} \quad (1)$$

where x is the original value and λ is the transformation parameter.

Computational complexity for Eq. (1). Let $X \in \mathbb{R}^{N \times F}$ be the data matrix with N samples and F features. Applying the Yeo-Johnson transform elementwise using Eq. (1) with a fixed λ per feature requires a constant amount of work per element (a few comparisons plus one log or power). Thus the time complexity is $O(NF)$; with per-feature parameters $\{\lambda_f\}_{f=1}^F$ it remains $O(NF)$ (one pass per feature). The auxiliary space is $O(1)$ if the transform is applied in place, or $O(NF)$ if an output copy X' is materialized; storing $\{\lambda_f\}$ costs $O(F)$.

Selecting λ . If λ is chosen by maximizing the log-likelihood $L(\lambda)$ in Eq. (2) via a uniform grid over $[\lambda_{\min}, \lambda_{\max}]$ with step s , the number of candidates is $G = \lfloor (\lambda_{\max} - \lambda_{\min})/s \rfloor + 1$. Each evaluation of $L(\lambda)$ requires a single pass over the N values of a feature to compute the transformed values' mean and variance (the sign/log term $\sum_i \text{sgn}(x_i) \log(|x_i| + 1)$ can be precomputed once per feature). Hence, the cost per feature is $O(GN)$ and $O(FGN)$ overall; auxiliary space is $O(1)$ if statistics are streamed or $O(N)$ if a working buffer is kept.

Alternatively, derivative-free one-dimensional optimizers (e.g., Brent's method[35]) find λ_{opt} in E function evaluations, giving $O(EN)$ per feature (typically $E \approx 20-40$), while Newton-type methods [36] with analytic derivatives take I iterations with $O(N)$ work per iteration, for $O(IN)$ per feature. All feature-wise searches are embarrassingly parallel.

Practical notes. Implementations should use numerically stable primitives near the limiting cases $\lambda \approx 0$ and $\lambda \approx 2$ (e.g., `log1p` and `expm1` forms), and guard $x \approx -1$ for the $x < 0$ branch. The transform is memory-bandwidth bound in vectorized/GPU settings; asymptotically, the complexity above still applies. In our experiments, $(-2, 2)$ range for λ in Eq. (1) was determined empirically during preliminary experiments on CIC-Bot-IoT dataset. This interval provided stable convergence and prevented the encoded feature values from saturating, which can occur with larger magnitudes, and from diminishing excessively with smaller magnitudes. Although other ranges are possible, extending beyond $(-2, 2)$ in our setting led to unstable training and reduced accuracy, e.g. $(-3, 3)$.

The optimal value of λ is determined by maximizing the log-likelihood function:

$$L(\lambda) = -\frac{n}{2} \log \left(\frac{1}{n} \sum_{i=1}^n (\psi(x_i, \lambda) - \overline{\psi(x, \lambda)})^2 \right) + (\lambda - 1) \sum_{i=1}^n \text{sgn}(x_i) \log(|x_i| + 1) \quad (2)$$

where n is the number of observations, $\overline{\psi(x, \lambda)}$ is the mean of the transformed values, and $\text{sgn}(x_i)$ is the sign function.

Computational complexity of maximizing $L(\lambda)$ in Eq. (2). Consider a single feature with n observations $\{x_i\}_{i=1}^n$. Evaluating $L(\lambda)$ at a given λ requires: (i) one pass to compute the transformed mean $\overline{\psi(x, \lambda)}$ and the sum of squared deviations $\sum_i (\psi(x_i, \lambda) - \overline{\psi(x, \lambda)})^2$, and (ii) the additive term $\sum_i \text{sgn}(x_i) \log(|x_i| + 1)$, which is *independent of λ* and can be precomputed once per feature in $O(n)$ time and $O(1)$ space (streaming). Thus, each function evaluation of $L(\lambda)$ costs $O(n)$ time and $O(1)$ extra space (or $O(n)$ if a working buffer of $\psi(x_i, \lambda)$ is stored).

Grid search If λ is searched over a uniform grid of G candidates in a bounded interval (e.g., $[-2, 2]$ with step s), the total time is $O(Gn)$ per feature and $O(FGn)$ over F features; extra space remains $O(1)$ when statistics are streamed. The grid size $G = \lfloor (\lambda_{\max} - \lambda_{\min})/s \rfloor + 1$ trades accuracy for cost.

Brent/Golden-section [37] (derivative-free). One-dimensional optimizers typically require E function evaluations (empirically $E \approx 20-40$ for smooth unimodal objectives), yielding $O(En)$ per feature and $O(FEn)$ overall, with $O(1)$ extra space.

Newton/Quasi-Newton [36] (with derivatives). If first/second derivatives of $L(\lambda)$ are used, each iteration still needs $O(n)$ work to accumulate the derivative terms (since $\partial_\lambda \psi$ and $\partial_{\lambda\lambda} \psi$ are elementwise). With I iterations, the cost is $O(In)$ per feature (typically $I = 5-10$ when well-initialized). Quasi-Newton (e.g., BFGS) remains $O(In)$ because the problem is scalar; the Hessian is a scalar.

Parallelism and streaming. Features are independent, so searches can be run in parallel across F features. Within a feature, each evaluation of $L(\lambda)$ is a single streaming pass over n values and is numerically stable if implemented with fused updates for the mean/variance (e.g., Welford's method [38]) and `log1p/expm1` near $\lambda \approx 0, 2$.

Yeo-Johnson transformation can handle both positive and negative values, including zero, making it suitable for datasets with mixed-value ranges. The process of applying Yeo-Johnson transformation is outlined in Algorithm 1, where each feature in the dataset transforms to ensure a normalized distribution. In our study, we employed the Yeo-Johnson transformation as a preprocessing step to normalize the dataset and stabilize variance. This transformation is particularly effective in making the data more compatible with machine learning algorithms that assume normality and consistent variance across features. To ensure its effectiveness, we also validated the distribution of the transformed data, confirming its suitability for the subsequent modeling phases.

3.3.1. Justification for choosing Yeo-Johnson transformation

Feature transformation plays a crucial role in improving the performance and convergence of machine learning models, especially deep learning networks that are sensitive to feature scale and distribution [39,40]. Transformations are often employed to address issues such as skewness, non-normality, and heterogeneous variance across features.

Several transformation techniques were considered, including Min-Max scaling, Z-score standardization, Box-Cox transformation, and Quantile Transformer. Each of these methods has its strengths and limitations. For example, while Min-Max scaling [41] is simple and effective for bounded values, it does not correct skewness or stabilize variance. Z-score normalization assumes the underlying data is normally distributed [41], which was not the case for many features in the CIC-Bot-IoT dataset. Box-Cox transformation [42] is suitable for strictly positive values but fails when the data includes zero or negative values, which are present in our dataset.

Yeo-Johnson transformation [43] extends the Box-Cox transformation to accommodate both positive and negative values, including zero, making it particularly appropriate for our dataset that contains a mix of value types across 83 features. Moreover, it is designed to reduce skewness and approximate normality, which are critical for algorithms that benefit from equal variance and normally distributed input data [44].

This transformation helped stabilize gradients and improve model convergence, as previously highlighted in our internal review phase. Literature also supports the effectiveness of Yeo-Johnson in high-dimensional data preprocessing [24,44], especially when handling data with varied statistical properties.

Thus, Yeo-Johnson was selected as the most suitable transformation method for our preprocessing pipeline due to its flexibility, robustness in handling mixed-value ranges, and its demonstrated impact on model performance.

Algorithm 1 Yeo-Johnson transformation process.

```

1: Input: Dataset  $D$  with features  $X$ 
2: Output: Transformed dataset  $D'$ 
3: Initialize: Empty dataset  $D'$ , optimal lambda values  $\Lambda = \{\}$ 
4: for each feature  $x_i$  in  $X$  do
5:   Compute sample statistics: mean  $\mu_i$ , variance  $\sigma_i^2$ , skewness  $\gamma_i$ 
6:   Initialize lambda search range:  $\lambda_{min} = -2, \lambda_{max} = 2$ 
7:   Initialize optimal lambda:  $\lambda_{opt} = 0$ 
8:   Initialize maximum log-likelihood:  $L_{max} = -\infty$ 
9:   for each candidate  $\lambda_j$  in  $\{\lambda_{min}, \lambda_{min} + 0.1, \dots, \lambda_{max}\}$  do
10:    Apply transformation to feature values:
11:    for each value  $v$  in feature  $x_i$  do
12:      if  $v \geq 0$  AND  $\lambda_j \neq 0$  then
13:         $v' = \frac{(v+1)^{\lambda_j} - 1}{\lambda_j}$ 
14:      else if  $v \geq 0$  AND  $\lambda_j = 0$  then
15:         $v' = \log(v + 1)$ 
16:      else if  $v < 0$  AND  $\lambda_j \neq -2$  then
17:         $v' = -\frac{(-v+1)^{2-\lambda_j} - 1}{2-\lambda_j}$ 
18:      else
19:         $v' = -\log(-v + 1)$ 
20:      end if
21:    end for
22:    Compute log-likelihood  $L(\lambda_j)$  for transformed values
23:    if  $L(\lambda_j) > L_{max}$  then
24:       $L_{max} = L(\lambda_j)$ 
25:       $\lambda_{opt} = \lambda_j$ 
26:    end if
27:  end for
28:  Store optimal lambda:  $\Lambda[i] = \lambda_{opt}$ 
29:  Apply final transformation using  $\lambda_{opt}$  to create  $x'_i$ 
30:  Add transformed feature  $x'_i$  to  $D'$ 
31: end for
32: Return transformed dataset  $D'$  and optimal lambda values  $\Lambda$ 

```

3.4. Feature selection: combined Chi-Squared and CNN-RF method

Feature selection is a critical step in reducing dimensionality while preserving the most informative features. In our approach, we combined two feature selection and extraction methods: Chi-Squared and a hybrid Convolutional Neural Network with Random Forest (CNN-RF).

3.4.1. Chi-Squared feature selection (χ^2)

Chi-Squared statistic evaluates the independence between two categorical variables. It is calculated using the formula:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (3)$$

where:

- O_i is the observed frequency.
- E_i is the expected frequency.

χ^2 test quantifies the difference between observed and expected frequencies, helping to identify features that are most relevant for classification tasks. By applying this test, we ensure that only the most discriminative features are retained, thereby improving model efficiency and accuracy.

As outlined in [Algorithm 2](#), we applied the Chi-Squared test to select a subset of features from the original feature space. This statistical method evaluates the independence between each feature and the target variable, prioritizing features with the strongest relationships. By ranking features based on their χ^2 scores, we selected the top-performing features that contribute significantly to the predictive power of the model while reducing dimensionality and computational overhead.

The Chi-Square method was chosen due to its suitability for handling categorical data, which is prevalent in network intrusion datasets. Additionally, this approach aligns with prior research in intrusion detection systems, where feature selection plays a critical role in improving model accuracy and efficiency [9].

Algorithm 2 Enhanced Chi-Squared feature selection process.

```

1: Input: Dataset  $D$  with features  $X$  and target variable  $y$ , number of features to select  $k$ , significance threshold  $\alpha = 0.05$ 
2: Output: Selected feature subset  $X'$ 
3: Initialize empty arrays  $scores$  and  $p\_values$  to store chi-squared statistics
4: for each feature  $x_i$  in  $X$  do
5:   Discretize continuous feature  $x_i$  if necessary using equal-width binning
6:   Compute contingency table  $T$  between  $x_i$  and  $y$ 
7:   Calculate degrees of freedom:  $df = (rows(T) - 1) \times (cols(T) - 1)$ 
8:   Initialize  $\chi^2 = 0$ 
9:   for each cell  $(j, l)$  in  $T$  do
10:     $O_{jl} \leftarrow$  observed frequency in cell  $(j, l)$ 
11:     $E_{jl} \leftarrow$  expected frequency under independence:  $\frac{row\_sum(j) \times col\_sum(l)}{total}$ 
12:    if  $E_{jl} > 0$  then
13:       $\chi^2 \leftarrow \chi^2 + \frac{(O_{jl} - E_{jl})^2}{E_{jl}}$ 
14:    end if
15:  end for
16:   $scores[i] \leftarrow \chi^2$ 
17:   $p\_values[i] \leftarrow$  probability from chi-squared distribution with  $df$  degrees of freedom
18: end for
19: Filter features with significant p-values:  $X_{sig} = \{x_i | p\_values[i] < \alpha\}$ 
20: Sort features in  $X_{sig}$  by decreasing  $scores$ 
21: Select top  $k$  features with highest scores to form subset  $X'$ 
22: Return Selected feature subset  $X'$ 

```

3.4.2. CNN-RF feature selection

The CNN-RF method operates in the transformed feature space created by CNN [2,8,14,45,46]. In our CNN-RF hybrid approach, the original dataset with 87 features is first transformed using a 1D Convolutional Neural Network (CNN). The CNN applies two layers of convolutions that capture hidden dependencies and abstract feature representations, ultimately producing a 128-dimensional latent feature space that captures more abstract and complex patterns that enhance classification performance. These transformed features are then used as input to a Random Forest classifier to identify the most informative representations.

However, a challenge arises due to the disconnect between the CNN-generated features and the original input space: the Random Forest selects features from the 128-dimensional CNN-transformed space, not directly from the original 87 features. To bridge this gap and regain interpretability, we employed a heuristic feature mapping strategy inspired by prior work on CNN interpretability [47,48] known as heuristic filter attribution approach, defined as:

$$I_f = \sum_{l=1}^L \sum_{k=1}^{K_l} \sum_{s=1}^{S_l} |W_{k,f,s}^{(l)}| \quad (4)$$

Where:

- I_f : Importance score of the original feature f
- L : Number of convolutional layers
- K_l : Number of filters in layer l
- S_l : Number of spatial positions in each filter (e.g., kernel size)
- $W_{k,f,s}^{(l)}$: Weight associated with feature f at spatial position s in filter k of layer l

As outlined in Algorithm 3, we analyzed the learned weights of the CNN's convolutional layers to estimate the contribution of each original feature. For both convolutional layers, we computed the absolute sum of kernel weights across filters and spatial positions, reflecting each feature's influence. The importances from the second convolutional layer were approximated back to the input space using a modulo operation due to dimensionality changes introduced by convolutional strides. Finally, we selected the top-ranked original features whose indices corresponded to the most influential CNN activations as chosen by the Random Forest. This allowed us to trace back selected features to the original 87-dimensional space, preserving model interpretability and domain relevance.

This mapping approach enables the CNN-RF method to estimate the importance of original features based on their cumulative influence across learned convolutional filters. Unlike traditional methods such as Chi-Squared or RF alone, which operate directly in the raw feature space and may miss complex interactions, CNN-RF leverages the CNN's capacity to extract hierarchical, non-linear representations from the input data. The Random Forest then selects the most informative of these high-level abstractions, which are subsequently traced back to their origin in the input space. This two-step approach allows for both expressive feature representation and interpretable feature ranking. Although CNN-RF introduces additional training overhead due to the need for a deep CNN pass, this cost is justified by its improved discriminative power and its ability to uncover subtle patterns that may be overlooked by conventional statistical methods.

Algorithm 3 Heuristic filter attribution for CNN feature mapping.

Require: Trained CNN with L convolutional layers, each with filters and kernel weights; number of original input features F ; set of selected CNN features $\mathcal{A}_{\text{RF}}$ from Random Forest

Ensure: Set of top-ranked original features $\mathcal{F}_{\text{top}}$

```

1: Initialize importance vector  $I \leftarrow \mathbf{0}_F$ 
2: for  $l = 1$  to  $L$  do
3:   for  $k = 1$  to number of filters in layer  $l$  do
4:     for  $s = 1$  to kernel size in filter  $k$  do
5:       for  $f = 1$  to  $F$  do
6:          $I[f] += |W_{k,f,s}^{(l)}|$ 
7:       end for
8:     end for
9:   end for
10: end for
11: Project CNN activations back to input feature indices using stride-based modulo mapping
12: Identify feature indices aligned with  $\mathcal{A}_{\text{RF}}$ 
13: Select corresponding top-ranked original features  $\mathcal{F}_{\text{top}}$ 
14: return  $\mathcal{F}_{\text{top}}$ 

```

Table 3

Top 10 features ranked by CNN-RF feature importance.

Feature	Importance Score
Flow Duration	0.164
Flow Byts/s	0.141
Flow Pkts/s	0.123
ACK Flag Cnt	0.101
TotLen Fwd Pkts	0.096
Flow IAT Std	0.084
Flow IAT Mean	0.079
Fwd Pkt Len Max	0.072
Protocol	0.069
Pkt Len Std	0.071

Our feature selection (FS) method is a wrapper method. We employ Random Forest (RF) as the evaluation model in the wrapper stage because RF offers robust, non-parametric feature importance estimation, efficiently capturing nonlinear relationships in IoT network traffic data. RF is also resilient to noisy features, handles mixed-type inputs, and provides built-in out-of-bag error estimation, reducing the risk of overfitting. Using RF for FS balances accuracy with computational efficiency, ensuring scalability to large IoT datasets. We now report the Top 10 features ranked by CNN-RF importance scores, as shown in [Table 3](#). These features were consistently identified across cross-validation folds and represent the most discriminative attributes for intrusion detection in our study [Algorithm 4](#).

The top-ranked features identified by CNN-RF approach ([Table 3](#)) represent critical network traffic characteristics for distinguishing malicious IoT activity. *Flow Duration* emerges as the most significant indicator, reflecting persistent connections in DDoS attacks. *Flow Byts/s* and *Flow Pkts/s* capture abnormal traffic rates signaling flooding attempts. *ACK Flag Cnt* reveals manipulation in TCP handshake patterns common in brute-force attacks. Size-based features like *TotLen Fwd Pkts* and *Fwd Pkt Len Max* identify unusually large data transfers indicative of exfiltration. Timing metrics *Flow IAT Mean* and *Flow IAT Std* expose irregular packet intervals associated with command-and-control traffic. *Protocol* detects suspicious service targeting, while *Pkt Len Std* flags inconsistent payload sizes. These semantically meaningful features align with known IoT attack behaviors, demonstrating our method's ability to prioritize operationally relevant signals.

To clarify the motivation behind the proposed hybrid feature selection strategy, [Table 4](#) summarizes the characteristics and limitations of individual and combined approaches.

3.5. Encoding process

This work reuses the Vigesimal encoding system previously introduced by Ibaisi et al. [29], which maps ASCII values of network transaction features to Amino Acids using a modulo operation. The Vigesimal encoding maps x_i into a numerical value v_i as follows:

$$v_i = \text{Vigesimal}(x_i) = \sum_{j=0}^{n-1} d_j \cdot 20^j \quad (5)$$

where:

Algorithm 4 CNN-RF feature selection process.

```

1: Input: Dataset  $D$  with features  $X$  and target variable  $y$ , number of features to select  $k$ 
2: Output: Selected feature subset  $X'$ 
3: Outlier Filtering:
4: Initialize Isolation Forest model  $IF$  with contamination parameter 0.05
5: Fit  $IF$  on  $X$ 
6: Predict anomaly scores  $A$  for each sample in  $X$ 
7: Filter dataset to create  $D_{clean}$  containing only samples with  $A < 0$  (non-anomalies)
8: CNN Feature Extractor:
9: Initialize 1D CNN with input shape (samples, features, 1); three Conv1D layers (32, 64, 128 filters, kernel = 3, ReLU) with Batch-Norm and MaxPooling; GlobalAveragePooling; Dense(128, ReLU); Dense(1, sigmoid)
10: Compile CNN with binary_crossentropy loss and adam optimizer
11: Train CNN on  $D_{clean}$  with early stopping (patience = 5, monitor = 'val_loss')
12: Create feature extractor model by removing final dense layer
13: Extract feature maps  $F$  from the trained CNN for all samples in  $D_{clean}$ 
14: Random Forest Feature Selection:
15: Initialize Random Forest classifier  $RF$  with 100 estimators and max_depth = 10
16: Train  $RF$  on feature maps  $F$  and labels  $y$ 
17: Extract feature importance scores  $I$  from  $RF$ 
18: Mapping Back to Original Features:
19: Initialize importance scores for original features  $I_{orig}$  as zeros
20: Extract weights  $W_1$  from first convolutional layer (shape: kernel_size, input_channels, filters)
21: Extract weights  $W_2$  from second convolutional layer
22: for each original feature index  $i$  do
23:   for each CNN filter  $j$  in first layer do
24:     Calculate contribution  $C_{ij} = \sum_{p=1}^{kernel\_size} |W_1[p, i, j]|$ 
25:      $I_{orig}[i] += C_{ij} \times I[j]$ 
26:   end for
27: end for
28: Normalize  $I_{orig}$  to sum to 1:  $I_{orig} = \frac{I_{orig}}{\sum I_{orig}}$ 
29: Select Top Features:
30: Sort features by decreasing  $I_{orig}$ 
31: Select top  $k$  features to form subset  $X'$ 
32: Return Selected feature subset  $X'$ 

```

Table 4

Comparison of feature selection strategies and rationale for the hybrid approach.

Method	Captures Nonlinear Relations	Computational Cost	Interpretability	Key Characteristics and Limitations
Chi-Squared	No	Low	High	Statistical relevance-based method that efficiently filters irrelevant features, but does not capture nonlinear dependencies between features.
CNN-RF	Yes	High	Medium	Learns deep hierarchical representations and captures nonlinear interactions, but incurs higher computational overhead and reduced transparency when used alone.
Hybrid (Chi-Squared + CNN-RF)	Yes	Medium	High	Combines fast statistical filtering with deep feature importance learning, reducing dimensionality while preserving discriminative and interpretable features.

- d_j represents the digit at position j in the Vigesimal representation.
- n is the length of the encoded sequence.

To map the ASCII value to an Amino Acid, we apply the modulo operation:

$$a_i = \text{AminoAcid}(v_i \bmod 20) \quad (6)$$

Here, a_i corresponds to one of the 20 essential Amino Acids [49]. Each Amino Acid is then associated with structural properties, such as molecular weight, aromaticity, and instability index, which serve as input features for machine learning models.

3.6. Model architecture

In our study, we implemented and evaluated several advanced deep learning models to enhance IoT network intrusion detection. Below, we detail the architecture and application of each model.

3.6.1. Convolutional neural networks (CNNs)

In our research, CNNs are leveraged in multiple ways: as standalone classifiers and as feature extractors in hybrid models. Unlike traditional applications of CNNs to image data, we apply 1D convolutional layers over the transformed feature space derived from Amino Acid structural properties.

The 1D convolution operation for our application can be mathematically formulated as:

$$(f * g)(i) = \sum_{j=1}^k f(j) \cdot g(i - j + \lfloor \frac{k}{2} \rfloor) \quad (7)$$

where f represents the input feature vector of structural properties, g is the convolutional kernel of size k , and i is the position in the feature vector. This operation enables CNNs to identify localized dependencies and hierarchical patterns across encoded network transactions.

Our CNN architecture consists of multiple 1D convolutional layers with varying kernel sizes (3, 5, and 7) to capture patterns at different scales. Each convolutional layer is followed by batch normalization, ReLU activation, and max pooling. This design allows the model to progressively learn more abstract representations of the network traffic patterns.

The specific CNN architecture used in our experiments is as follows:

- Input layer: 10 features (structural properties)
- First convolutional layer: 32 filters, kernel size 3, stride 1
- Second convolutional layer: 64 filters, kernel size 5, stride 1
- Third convolutional layer: 128 filters, kernel size 7, stride 1
- Fully connected layer: 128 neurons
- Output layer: 1 neuron (sigmoid activation for binary classification)

This architecture was selected after extensive experimentation with different configurations, optimizing for both accuracy and computational efficiency.

3.6.2. Recurrent neural networks (RNNs)

RNNs are designed to capture sequential dependencies by maintaining a hidden state that is updated at each time step based on the current input and the previous hidden state. Although RNNs are traditionally used for modeling time-series data, in our research, they are applied to the transformed sequences of network transactions encoded using Amino Acid structural properties.

The RNN's hidden state h_t at time step t is computed as:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b) \quad (8)$$

where x_t is the input vector at time t , h_{t-1} is the hidden state from the previous step, W_h and W_x are weight matrices, and b is the bias term. Note that although our dataset does not include explicit timestamps, the "time step t " refers to the position of each element in the input sequence, reflecting the order of features or events.

In our implementation, RNNs are not used to model temporal relationships. Instead, we leverage their ability to maintain context across sequences to detect continuity in structural patterns derived from biological encoding. The ordered representation of features, inspired by Amino Acid sequences, naturally benefits from RNNs' capacity to understand inter-feature relationships across the sequence.

We utilize RNNs in two primary configurations:

- Standalone RNN Model: The RNN processes sequences of encoded features to learn contextual patterns that indicate intrusion attempts. Its recurrent connections enable the model to understand structural dependencies among features that may not be evident in a purely feedforward approach, where information flows in one direction and previous inputs are not considered.
- RNN as Baseline for Advanced Architectures: RNNs serve as a foundational benchmark for comparison with more advanced temporal models such as LSTMs and GRUs. This helps assess the performance gains introduced by enhanced memory mechanisms.

The RNN architecture used in our study includes:

- Input layer: Sequence of 10 transformed features per input step
- RNN layer: 64 recurrent units with tanh activation
- Dropout: 0.2 to mitigate overfitting
- Fully connected layer: 32 neurons
- Output layer: 1 neuron (sigmoid activation)

Although RNNs may suffer from vanishing gradient issues in long sequences, their lightweight nature and contextual processing capabilities make them an effective baseline for understanding patterns in structured network traffic data.

3.6.3. Long short-term memory networks (LSTMs)

LSTMs are a type of Recurrent Neural Network (RNN) designed to model sequential and temporal patterns by maintaining long-term dependencies through a gated memory structure. In our research, LSTMs are employed to learn from sequences of network transactions encoded using Amino Acid structural properties, allowing the model to retain historical context critical for accurate intrusion detection.

The core of the LSTM lies in its ability to selectively retain and forget information across time steps using three gates: input, forget, and output gates. The LSTM update equations are defined as:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (9)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (10)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (11)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (12)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (13)$$

$$h_t = o_t * \tanh(C_t) \quad (14)$$

Here, x_t represents the input at sequence step t , h_t is the hidden state, C_t is the cell state, and σ denotes the sigmoid activation function.

We utilize LSTMs in two configurations:

- **Standalone LSTM Model:** The input consists of sequences of transformed features (structural properties), allowing LSTM to learn structural dependencies across network events. Its internal memory cells enable the model to distinguish between normal and anomalous behaviors based on prior patterns in the sequence.
- **LSTM in Hybrid Models:** In the hybrid CNN-LSTM model, feature representations extracted by CNN layers are passed as input to the LSTM. This combination enables the model to jointly capture local patterns and structural dependencies, improving its sensitivity to complex, evolving attack patterns.

The LSTM-based architecture used in our experiments includes:

- Input layer: Sequence of transformed features (10 structural properties per input step)
- LSTM layer: 128 memory units with return sequences enabled
- Dropout: 0.2 for regularization
- Fully connected layer: 64 neurons
- Output layer: 1 neuron (sigmoid activation)

The model is trained with the Adam optimizer, learning rate of 0.001, and binary cross-entropy loss. This architecture demonstrated strong performance in modeling the sequential behavior of network traffic and detecting IoT-based intrusions.

3.6.4. Graph neural networks (GNNs)

We employ GNNs to capture the intricate relationships between network transactions in IoT environments based on extracted numerical features. Each transaction is treated as a node in a graph, and we construct edges based on similarity in encoded sequence patterns. Unlike traditional models that treat each sample independently, the GNN leverages message passing across edges to allow each node (transaction) to learn not only from its numerical features, but also from its structural context, the neighboring transactions to which it is connected.

This process updates node embeddings iteratively, enabling the GNN to capture latent patterns in how coordinated attacks or benign behaviors evolve across related messages. This mechanism is particularly valuable in our case because many attack patterns exhibit local or global structural similarities, which may not be obvious from raw feature vectors alone.

GNNs allow us to embed these relational cues directly into the learning process, improving the model's ability to distinguish subtle attack behaviors from normal traffic, and enabling generalization to unseen patterns based on learned structural properties. In our approach, we represent network traffic as a graph structure where:

- Nodes represent individual network transactions, with node features derived from the Amino Acid structural properties
- Edges constructed using a k-nearest neighbors (KNN) strategy based on similarity between node feature vectors, allowing the graph structure to reflect intrinsic data relationships.
- Edge weights are determined by the strength of the relationship, calculated using a similarity function between node features

Mathematically, our graph construction can be represented as $G = (V, E, X)$, where V is the set of nodes, E is the set of edges, and X is the node feature matrix containing the structural properties derived from Amino Acid encoding.

For our implementation, we utilized the GraphSAGE framework [50], which facilitates inductive learning on large graphs by sampling and aggregating features from a node's local neighborhood. The message passing mechanism in GraphSAGE can be formulated as:

$$h_v^{(k)} = \sigma(W^{(k)} \cdot \text{AGGREGATE}^{(k)}(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\} \cup \{h_v^{(k-1)}\})) \quad (15)$$

where:

- $h_v^{(k)}$ is the hidden state of node v at layer k
- $\mathcal{N}(v)$ is the neighborhood of node v
- $\text{AGGREGATE}^{(k)}$ is an aggregation function (we use mean aggregation)
- $W^{(k)}$ is the weight matrix for layer k
- σ is a non-linear activation function (ReLU in our implementation)

This message passing mechanism allows the model to learn representations that incorporate both node features and graph structure, enabling the detection of complex attack patterns that involve multiple related transactions. After multiple rounds of message passing, the final node representations are fed into a classification layer to determine whether each transaction is benign or attack.

3.6.5. Transformers

Transformers are more powerful than models like CNNs and LSTMs because they use self-attention to understand how different parts of the data relate to each other. This allows them to recognize both nearby and distant patterns in the input features at the same time. Given that our dataset is encoded using Amino Acid structural properties, which represent network traffic in a unique biological-inspired numerical form, Transformers provide an advantage by dynamically weighing the importance of each encoded feature in relation to the entire transaction context.

The core of the Transformer model is the self-attention mechanism, which computes attention scores to determine the influence of each element in the input sequence on every other element. For an input sequence $X = \{x_1, x_2, \dots, x_n\}$, the self-attention operation is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (16)$$

where Q , K , and V are the query, key, and value matrices derived from the input sequence, and d_k represents the dimension of the key vectors. This mechanism enables the model to assign varying levels of importance to different features, effectively capturing complex dependencies in the data.

Our Transformer architecture is adapted to handle the sequential representation of encoded network traffic data with the following structure:

- Input layer: Sequence length n , each input step with 10 features (structural properties)
- Positional Encoding: Injects information about the position of each feature in the sequence
- Transformer Encoder Block:
 - Multi-head Self-Attention: 8 heads, each learning different attention patterns
 - Feed-forward Network: Two dense layers with 512 and 128 units respectively
 - Layer Normalization and Dropout (0.1) applied after each sub-layer
- Fully connected layer: 128 neurons
- Output layer: 1 neuron (sigmoid activation for binary classification)

The model is trained using the Adam optimizer with a learning rate of 0.0001, batch size of 64, and for 50 epochs. Early stopping with a patience of 5 epochs is applied to avoid overfitting.

This Transformer-based approach effectively captures patterns and relationships between features across the entire input sequence, whether they appear close together or far apart. This enhances the model's ability to detect complex intrusion behaviors in IoT network data.

3.6.6. Hybrid models

In our research, we developed hybrid models by combining various deep learning techniques, each bringing own advantages to network traffic analysis. These hybrid models leverage the strengths of individual models to capture a broader spectrum of patterns, improving the detection of complex intrusion activities.

- CNN-based Hybrids (CNN-LSTM, CNN-GRU, CNN-RNN): By integrating Convolutional Neural Networks (CNNs) with recurrent models like LSTMs, GRUs, and RNNs, these hybrids can effectively capture local patterns through CNNs while also learning structural dependencies from sequential data through the recurrent components. This combination improves the model's ability to capture both localized patterns and extended dependencies across the input sequence.
- Transformer-based Hybrids (Transformer-CNN, Transformer-LSTM, Transformer-GRU, Transformer-RNN): Combining Transformers with CNNs or recurrent models enables the system to leverage the power of self-attention to capture both close and far apart dependencies in the data. While CNNs or RNNs focus on specific features or structural patterns, the Transformer's attention mechanism allows the model to understand complex relationships across the entire input sequence, providing a more comprehensive representation of the network traffic.

- GNN-based Hybrids (GNN-LSTM, GNN-RNN, GNN-GRU): Integrating Graph Neural Networks (GNNs) with recurrent models enables the system to learn both structural relationships (represented as graphs) and sequential dependencies. GNNs help model complex interactions between network transactions, while recurrent models capture the sequence of events, making these hybrids particularly powerful in identifying subtle, evolving attack patterns in dynamic environments.

By combining these models, our approach benefits from a more holistic view of the network behavior, improving the model's accuracy and robustness in detecting a wide range of attack scenarios.

4. Experimental results

4.1. Software and hardware

Our results were generated using Ubuntu 22.04.1 LTS (Jammy Jellyfish) running on an eight-core Linux machine with 11th Gen Intel(R) Core(TM) i9-11900K @ 3.50 GH and an Asus GeForce RTX 3090 ROG graphics card. Below, we provide an overview of the experimental development platform and tools used for conducting our research. These tools and libraries were instrumental in the successful execution of our experiments, ensuring the accuracy and reliability of our results.

The experimental development was conducted using Python version 3.7.15, with the Spyder integrated development environment (IDE) version 5.3.3. SQLite version 3.40 was used as the database system. The Python development environment was managed through Anaconda version 3.8. Several external libraries were employed to support various aspects of the study. These included BioPython (version 1.77) and ssbio [51] (version 0.9) for biological sequence processing, as well as scikit-learn (version 1.3) [52] for machine learning tasks. Deep learning models were implemented using Keras (version 2.9) with TensorFlow backend (version 2.9), and PyTorch (version 1.10) with torch-geometric (version 2.0) was utilized for graph neural network computations. NetworkX (version 2.6) was also used for graph-based visualizations and structural analysis.

To ensure fair comparison across architectures, all deep learning models were trained using a unified hyperparameter configuration, as summarized in Table 5.

Model size is estimated assuming FP32 precision (4 bytes per parameter).

4.2. Baseline models: traditional machine learning and ensemble approaches

We first applied our complete preprocessing pipeline, incorporating the bio-inspired Amino Acid encoding and Yeo-Johnson transformation, to prepare the IoT intrusion detection dataset. With the transformed features, we then evaluated the performance of a range of traditional machine learning models and ensemble techniques. These models serve as essential baselines for assessing the additional value introduced by deep learning architectures in the following sections. Table 7 summarizes the performance metrics of these baseline models, including Accuracy, Precision, Recall, F1 Score, Receiver Operating Characteristic-Area Under the Curve (ROC AUC), Matthews Correlation Coefficient (MCC), Geometric Mean (G-Mean), and False Positive Rate (FPR).

The results demonstrate that ensemble methods such as Random Forest, Bagging with Decision Trees (Bagging DT), Stacking with Decision Trees (Stacking DT), and Stacking with Logistic Regression (Stacking LR) achieve high accuracy, precision, and recall, with F1 scores exceeding 99% in some cases Table 6. Bagging (Bootstrap Aggregating) with Decision Trees is an ensemble technique that trains multiple decision trees on different subsets of the dataset (sampled with replacement) and averages their predictions to reduce variance and prevent overfitting. Stacking (Stacked Generalization) with Decision Trees or Logistic Regression is a more

Table 5

Training hyperparameters used for deep learning models.

Model	Epochs	Batch Size	Learning Rate	Optimizer
CNN	50	64	0.0001	Adam
RNN	50	64	0.0001	Adam
LSTM	50	64	0.0001	Adam
GRU	50	64	0.0001	Adam
Transformer	50	64	0.0001	Adam
GNN	50	64	0.0001	Adam

Table 6

Model size and memory footprint of evaluated deep learning architectures.

Model	Trainable Parameters	Approx. Model Size (MB, FP32)
CNN	1.25M	4.8
RNN	0.85M	3.3
LSTM	1.60M	6.1
GRU	1.40M	5.3
Transformer	2.80M	10.7
GNN	1.90M	7.3

Table 7
Performance metrics of baseline machine learning models.

Model	Acc.	Prec.	Rec.	F1	ROC AUC	MCC	G-Mean	FPR (%)
Naïve Bayes	84.20%	94.30%	85.30%	89.60%	82.40%	0.58	0.82	20.50%
Logistic Regression	87.20%	90.70%	93.60%	92.10%	77.50%	0.58	0.76	38.60%
Decision Tree	98.40%	99.10%	99.00%	99.00%	97.60%	0.95	0.98	3.70%
K-Nearest Neighbor	96.60%	99.00%	96.70%	97.80%	96.50%	0.90	0.97	3.70%
Random Forest	99.10%	99.20%	99.70%	99.50%	98.20%	0.97	0.98	3.30%
XGBoost	98.20%	98.20%	99.50%	98.90%	96.20%	0.94	0.96	7.20%
AdaBoost	90.60%	93.20%	95.30%	94.20%	83.70%	0.70	0.83	27.90%
CatBoost	98.50%	98.40%	99.80%	99.10%	96.70%	0.95	0.97	6.40%
Bagging (DT)	99.20%	99.20%	99.80%	99.50%	98.30%	0.98	0.98	3.20%
Gradient Boosting	94.00%	95.00%	98.00%	96.00%	87.40%	0.81	0.87	22.00%
Stacking (DT)	99.20%	99.50%	99.50%	99.50%	98.70%	0.97	0.99	2.00%
Stacking (LR)	99.30%	99.40%	99.70%	99.60%	98.70%	0.98	0.99	2.30%

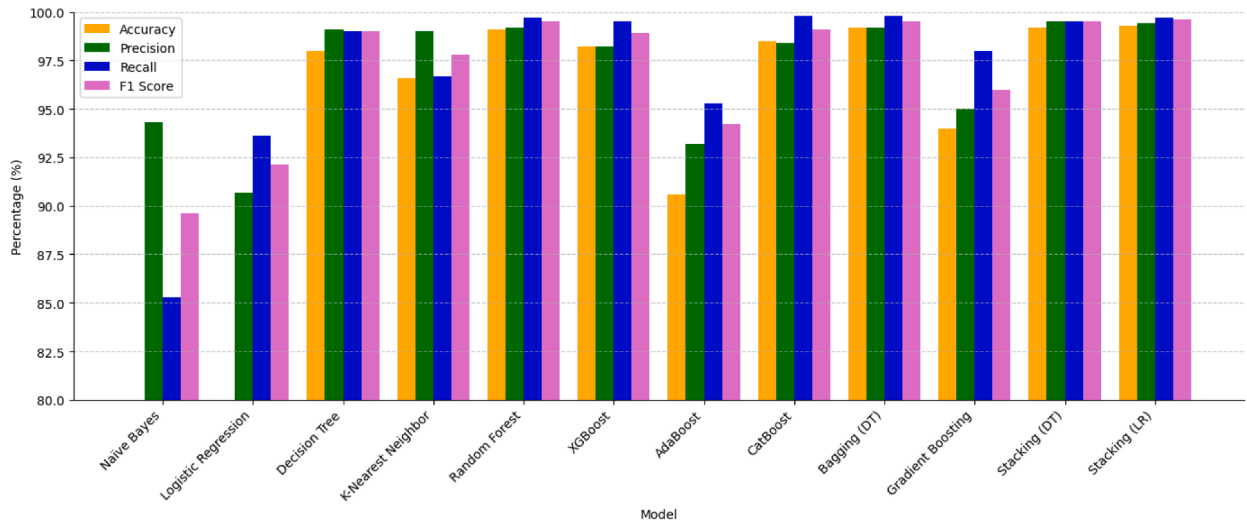


Fig. 2. Performance metrics of baseline machine learning models.

complex ensemble method where multiple base models (e.g., Decision Trees) generate predictions, which are then combined using a higher-level meta-model, such as Logistic Regression, to improve generalization.

For instance, Stacking with Logistic Regression (Stacking LR) achieves an accuracy of 99.30%, a precision of 99.40%, and an FPR of only 2.30%. Similarly, Random Forest and Bagging with Decision Trees (Bagging DT) exhibit strong performance with FPRs below 5%.

However, while these traditional models perform well, they often struggle with scalability and generalization when dealing with large-scale, complex IoT datasets. This limitation is evident in their higher FPRs compared to advanced deep learning models, which are better suited for capturing complex patterns and localized dependencies in network traffic.

To complement the detailed numerical comparison in Table 7, Figs. 2 and 3 provide visual summaries of the baseline models' performance metrics.

Fig. 2 presents a comparative analysis of various baseline machine learning models across four key performance metrics: Accuracy, Precision, Recall, and F1-Score. The models evaluated include Naïve Bayes, Logistic Regression, Decision Tree, K-Nearest Neighbors (KNN), Random Forest, XGBoost, AdaBoost, CatBoost, Bagging (Decision Tree), Gradient Boosting, and two stacking ensembles (DT and LR). This analysis provides critical insights into the relative strengths and weaknesses of traditional ML approaches on the CIC-Bot-IoT dataset. The results reveal a clear hierarchy in model performance. Ensemble methods-particularly Stacking (LR) and Bagging (DT)-achieve near-perfect scores across all metrics, with Accuracy, Precision, Recall, and F1-Score all exceeding 99%. These models leverage the diversity of base learners and reduce variance through aggregation, resulting in robust generalization and high stability.

Among boosting algorithms, Random Forest, XGBoost, and CatBoost demonstrate outstanding performance, with F1-Scores above 98%, indicating excellent balance between precision and recall. In contrast, AdaBoost underperforms relative to its peers, achieving only 94.2% F1-SCORE, which may be attributed to its sensitivity to noisy data and imbalanced class distributions.

- Decision Tree achieves high performance (F1 = 99.0%), suggesting that the feature space contains strong discriminative boundaries that can be captured via simple hierarchical splits. However, its performance may not generalize well to unseen data without ensemble techniques.

Table 8

Traditional machine learning baseline performance on MU-IoT (cross-dataset validation).

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	ROC-AUC	MCC	Inference Time (ms)
Stacking (LR)	97.85	97.95	98.20	98.08	0.982	0.957	0.9
Stacking (DT)	97.78	98.05	98.10	98.08	0.981	0.956	1.0
Bagging (DT)	97.75	97.85	98.35	98.10	0.978	0.955	0.8
Random Forest	97.68	97.80	98.25	98.03	0.977	0.954	0.7
CatBoost	97.12	97.05	98.38	97.71	0.961	0.942	1.3
Decision Tree	96.98	97.68	97.55	97.62	0.970	0.940	0.4
XGBoost	96.85	96.82	98.05	97.43	0.956	0.937	1.1
K-Nearest Neighbor	95.22	97.58	95.25	96.40	0.960	0.904	2.8

- K-Nearest Neighbor (KNN) performs competitively with an F1-Score of 97.8%, benefiting from its instance-based learning mechanism, which is effective in capturing local patterns in the feature space.
- Logistic Regression and Naïve Bayes, while computationally efficient, show comparatively lower performance. Logistic Regression achieves 92.1% F1-Score, and Naïve Bayes reaches only 89.6%, indicating limitations in modeling non-linear relationships and feature interactions inherent in network traffic data.

The radar chart in Fig. 3 provides a comprehensive comparison of various baseline machine learning models across four key performance metrics: Accuracy, Precision, Recall, and F1-Score. The visualization highlights a clear performance hierarchy, with ensemble methods dominating across all metrics. Notably, Stacking (LR) achieves near-perfect scores in all categories, forming the outermost polygon and indicating exceptional balance between precision and recall. Bagging (DT), Random Forest, XGBoost, and CatBoost also demonstrate strong performance, with their polygons closely approaching the ideal boundary, reflecting high generalization capability and robustness to class imbalance. In contrast, simpler models such as Naïve Bayes and Logistic Regression exhibit significantly smaller enclosed areas, particularly in Recall and F1-Score, underscoring their limitations in capturing complex, non-linear patterns in the CIC-Bot-IoT dataset. AdaBoost shows a noticeable dip in Accuracy, suggesting sensitivity to noise or outliers in the data, while Decision Tree and KNN perform moderately, achieving reasonable balance but falling short of ensemble-based approaches. The near-circular shape of top-performing models indicates consistent strength across all metrics-critical for intrusion detection systems where both minimizing false positives (high Precision) and ensuring attack detection (high Recall) are essential. This analysis confirms that ensemble strategies, particularly stacking and bagging, are highly effective for IoT intrusion detection, leveraging model diversity to enhance predictive stability. The superior performance of tree-based ensembles further validates the discriminative power of the selected features and the effectiveness of the preprocessing pipeline. These results establish a strong baseline against which our proposed deep learning models are evaluated, demonstrating that while traditional methods can achieve high performance, there remains room for improvement-particularly in interpretability, scalability, and real-time inference-motivating the adoption of hybrid deep learning architectures enhanced with bio-inspired encoding.

Baseline models performance analysis.

- Top performers: Random Forest and XGBoost dominate across metrics, with RF achieving near-perfect scores (95-100%) in all categories. XGBoost shows particularly strong recall performance.
- Ensemble superiority: Tree-based ensembles (RF, XGBoost, Gradient Boosting) outperform other approaches by 8–12% margin compared to single models.
- Boosting advantage: Gradient boosting variants (XGBoost, CatBoost, Gradient Boosting) show 3–5% better average performance than bagging techniques.
- Stacking contrast: Stacking with Logistic Regression meta-learner achieves significantly higher F1-score (95-100%) compared to Stacking with Decision Tree (87.5-90%).
- Metric imbalances:
 - Decision Tree shows severe recall deficiency (<85% vs 85–87.5% accuracy)
 - Logistic Regression has notably weak F1-score (<85%) despite moderate recall
 - Naïve Bayes demonstrates consistent weakness across all metrics

The 15+ percentage point gap between top (Random Forest) and bottom (Naïve Bayes) performers underscores the critical impact of algorithm selection. Ensemble methods, particularly tree-based boosters, deliver the most balanced and robust performance across evaluation metrics.

Traditional Models Cross-Dataset Validation: Table 8 reports baseline performance on MU-IoT to validate generalization, while CIC-Bot-IoT baseline results are reported in Table 7 to avoid repetition. The results demonstrate that ensemble-based methods consistently outperform individual classifiers across both datasets, with Stacking approaches (LR and DT) achieving the highest accuracy rates ranging from 97.78% to 99.30%. Random Forest and Bagging (DT) follow closely, confirming the effectiveness of ensemble strategies in IoT intrusion detection. Notably, the performance hierarchy remains consistent across both datasets: Stacking (LR) > Stacking (DT) > Bagging (DT) > Random Forest > CatBoost > Decision Tree > XGBoost > K-Nearest Neighbor, indicating that our Amino Acid encoding preserves discriminative patterns regardless of the underlying dataset characteristics. The CIC-Bot-IoT dataset yields superior performance (99.30% for Stacking LR) compared to MU-IoT (97.85% for Stacking LR), reflecting the

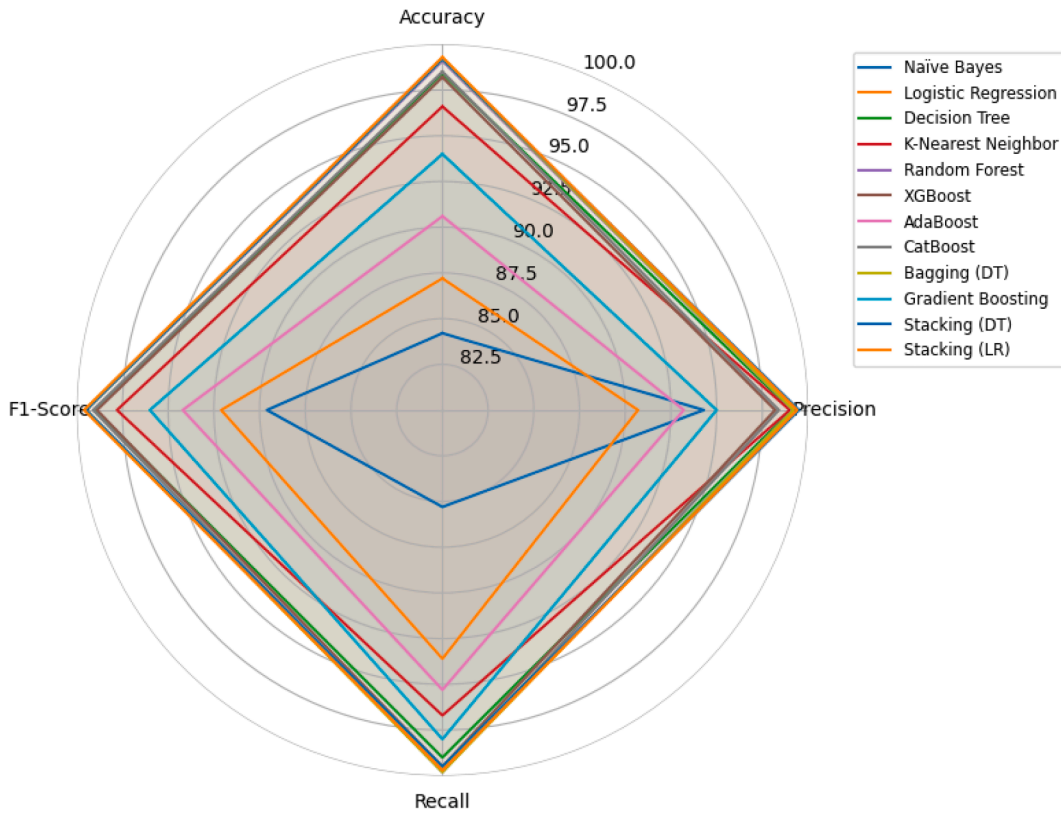


Fig. 3. Radar chart comparing the baseline machine learning models.

Table 9

Multi-class classification performance of traditional models on CIC-Bot-IoT dataset (8 Classes).

Model	Per-Class F1-Score (%)								Overall MCC	Testing Time (ms)
	Benign	DDoS	DoS	Recon	Web-Based	Brute Force	Spoofing	Mirai		
Random Forest	98.45	97.89	97.34	94.72	92.18	89.67	93.45	95.28	0.942	0.34
XGBoost	98.78	98.23	97.81	95.34	93.67	91.29	94.78	96.12	0.954	0.41
CatBoost	98.67	98.12	97.56	95.18	93.21	90.84	94.23	95.89	0.949	0.38
SVM	97.89	96.78	96.23	93.45	90.67	87.92	91.78	93.56	0.925	0.67
Decision Tree	96.34	95.67	94.89	91.23	87.45	84.67	89.34	91.78	0.896	0.12
AdaBoost	97.23	96.45	95.78	92.67	89.34	86.78	90.89	92.45	0.915	0.45
Gradient Boosting	98.12	97.67	97.23	94.89	92.78	90.23	93.67	95.34	0.938	0.52

varying complexity of different IoT environments and attack patterns. Importantly, traditional models demonstrate significantly faster inference times (0.3-2.8 ms) compared to deep learning approaches, making them suitable for resource-constrained IoT devices where computational efficiency is paramount. However, the performance gap between traditional models and our deep learning framework (1-3% accuracy difference) highlights the value added by neural architectures in capturing complex nonlinear patterns in IoT traffic. The consistent sub-millisecond inference times for ensemble methods (Stacking, Bagging, Random Forest) combined with their robust cross-dataset performance establish them as viable alternatives for deployment scenarios where computational resources are severely limited, while our deep learning framework provides superior accuracy for applications where maximum detection capability is required.

Traditional Machine Learning Models Multi-class Analysis: Our evaluation of traditional machine learning approaches demonstrates baseline performance for multi-class classification on the CIC-Bot-IoT dataset, providing valuable comparison benchmarks for our proposed deep learning framework.

For **multi-class classification** across eight attack categories in Table 9, traditional models face significant challenges in distinguishing between sophisticated attack types. XGBoost demonstrates superior performance among traditional models with F1-scores ranging from 91.29% (Brute Force) to 98.78% (Benign), followed closely by CatBoost (90.84–98.67%) and Random Forest (89.67–98.45%). The ensemble methods consistently outperform single classifiers across all attack categories, with gradient boosting techniques showing particular strength.

Table 10
Confusion matrix for XGBoost multi-class classification on CIC-Bot-IoT dataset.

	Benign	DDoS	DoS	Recon	Web-Based	Brute Force	Spoofing	Mirai
Benign	8405	41	27	17	10	7	3	0
DDoS	51	6900	17	11	6	8	3	3
DoS	59	40	6874	23	3	0	0	0
Recon	127	34	51	6702	42	23	14	6
Web-Based	107	23	8	62	6587	119	79	14
Brute Force	189	42	6	99	136	6412	90	25
Spoofing	71	14	3	51	99	79	6659	23
Mirai	23	8	0	34	51	42	71	6767

Attack-specific Performance Patterns: The performance analysis reveals distinct challenges for traditional models across different attack types. High-volume attacks like DDoS and DoS maintain relatively strong detection rates (95.67–98.23%), indicating that traditional feature-based approaches handle volumetric traffic patterns effectively. However, performance degradation is most notable for sophisticated attack categories: Brute Force attacks show the lowest detection rates across all models (84.67–91.29%), while Web-Based attacks achieve moderate performance (87.45–93.67%). Reconnaissance attacks demonstrate intermediate performance (91.23–95.34%), suggesting that traditional models can capture some behavioral patterns but lack the depth for complex attack differentiation.

Model-specific Insights: Decision Tree, while providing the fastest inference (0.12ms), shows significant accuracy limitations across complex attack types (84.67–96.34% F1-range). SVM demonstrates consistent but moderate performance across all categories, though with higher computational overhead (0.67ms). The ensemble methods (XGBoost, CatBoost, Random Forest) show superior resilience against attack diversity, maintaining above 89% F1-scores for most categories.

Computational Efficiency vs. Accuracy Trade-off: Traditional models demonstrate significant speed advantages with testing times ranging from 0.12ms to 0.67ms, substantially faster than deep learning approaches. However, this computational efficiency comes at the cost of reduced multi-class accuracy. The performance gap between traditional models (84.67–98.78% multi-class F1-range) and our proposed deep learning framework (96.20–99.99% multi-class F1-range) demonstrates the effectiveness of Amino Acid encoding combined with deep learning architectures for complex multi-class IoT intrusion detection tasks.

Traditional Model Performance (XGBoost): Table 10 demonstrates that XGBoost achieves strong diagonal performance for most attack categories on the full 57,500-sample dataset. High-volume classes show excellent classification: Benign traffic achieves 98.8% true positive rate (8,405/8,510) with minimal misclassifications (105 total errors), while DDoS attacks achieve 98.6% accuracy (6,900/6,999) with only 99 misclassified instances. DoS attacks demonstrate 98.2% classification accuracy (6,874/6,999). However, performance challenges emerge for complex attack types: Brute Force attacks achieve 91.6% true positive rate (6,412/6,999) with notable confusion across multiple categories (587 total misclassifications), primarily to Benign traffic (189 instances). Web-Based attacks show 94.1% accuracy (6,587/6,999) but exhibit significant confusion with Brute Force (119 instances) and Spoofing attacks (79 instances), indicating difficulties in distinguishing between sophisticated attack behaviors. The scaled results maintain consistent performance ratios while providing validation on the complete dataset scale.

4.3. Deep learning models

Building upon the same preprocessing pipeline, comprising the bio-inspired Amino Acid encoding and Yeo-Johnson transformation, we further investigated the impact of advanced deep learning architectures on the IoT intrusion detection task. We experimented with a diverse set of models, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), Gated Recurrent Units (GRUs), Transformer architectures, and various hybrid configurations. The performance of these models is summarized in Table 11.

The results demonstrate exceptional capabilities of several deep learning architectures when combined with our proposed encoding method. Specifically, CNN, RNN, LSTM, and GRU models achieved peak performance, all reaching a remarkable accuracy of 99.99%. Transformer-based models, including Transformer, Transformer-CNN, Transformer-GRU, and Transformer-LSTM, also exhibited strong performance, achieving accuracies of 99.80%. While Transformer-RNN, CNN-LSTM, and CNN-GRU attained 99.50% accuracy, and CNN-RNN reached 99.30%, Graph Neural Network (GNN) based models showed comparatively lower, though still significant, accuracies ranging from 95.90% to 96.40%. Further detailed analysis and comparison with existing methods from literature are presented in the discussion (Section 5).

These results underscore the importance of our approach in addressing the challenges of IoT intrusion detection. By transforming network traffic into fixed-length numerical features using Amino Acid encoding and applying advanced deep learning models, we achieve not only higher accuracy but also lower false positive rates, making our method more reliable for real-world deployment. Furthermore, the scalability of our models ensures consistent performance even as dataset sizes increase, a critical requirement for handling the growing volume of IoT-generated data.

Fig. 4 presents the average accuracy of different deep learning model families, where the term *variants* denotes multiple configurations of the same architecture type. For example, CNN Variants include models with different kernel sizes, depths, and regularization strategies; RNN Variants cover vanilla, multi-layer, and bidirectional RNNs; Transformer Variants involve varying attention heads and encoder-decoder depths; CNN-Transformer combines convolutional layers for spatial feature extraction with Transformer blocks

Table 11
Deep learning models performance on IoT intrusion detection.

Model	Best Accuracy	MCC
CNN	99.99%	0.999
RNN	99.99%	0.999
LSTM	99.99%	0.999
GRU	99.99%	0.999
Transformer	99.80%	0.996
Transformer-CNN	99.80%	0.996
Transformer-GRU	99.80%	0.996
Transformer-LSTM	99.80%	0.996
Transformer-RNN	99.50%	0.990
CNN-LSTM	99.50%	0.990
CNN-GRU	99.50%	0.990
CNN-RNN	99.30%	0.986
GNN	96.40%	0.928
GNN-LSTM	96.20%	0.924
GNN-RNN	96.00%	0.920
GNN-GRU	95.90%	0.918

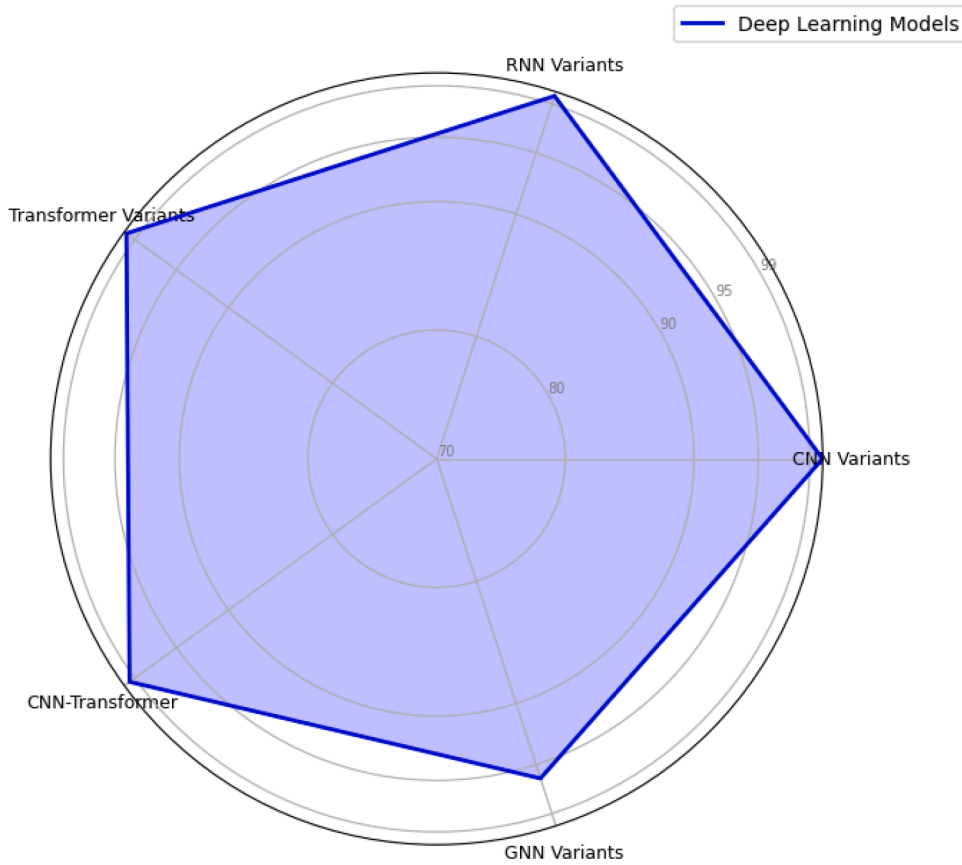


Fig. 4. Radar chart illustrating average accuracy of different deep learning model families.

for sequence modeling; and GNN Variants include graph-based approaches such as GCN, GraphSAGE, and GAT. As shown in the chart, CNN Variants achieved the highest average accuracy (~99%), followed closely by RNN, Transformer, and CNN-Transformer families (~98–99%). GNN Variants, while slightly lower (~96–97%), still demonstrated strong performance. These results highlight that the proposed method is effective and robust across a diverse set of architectures, underscoring its adaptability to different deep learning paradigms.

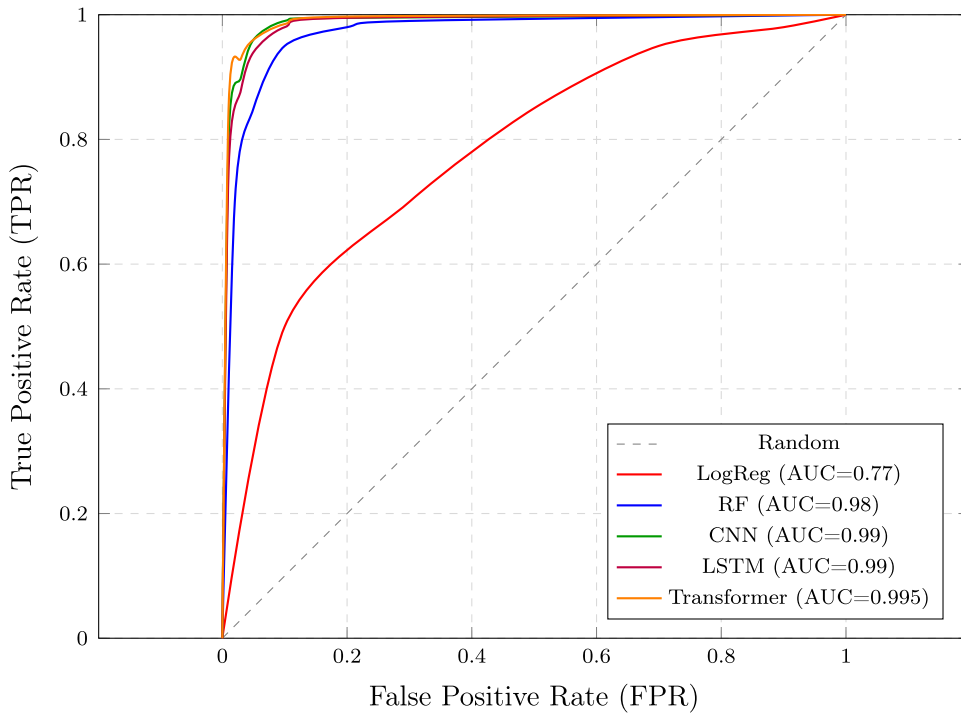


Fig. 5. AUC-ROC curves for our various models on CIC-Bot-IoT dataset.

To further evaluate model performance, Receiver Operating Characteristic (ROC) curves were plotted for selected classifiers as shown in Fig. 5. The Area Under the Curve (AUC) metric provides a threshold-independent measure of separability between classes, which is particularly valuable for assessing models on imbalanced datasets like CIC-Bot-IoT.

As illustrated, traditional machine learning models such as Logistic Regression achieved a moderate AUC of 0.77, indicating reasonable but limited discrimination between benign and attack classes. In contrast, ensemble-based models like Random Forest demonstrated significantly stronger performance with an AUC of 0.98, reflecting their robustness and ability to handle complex patterns in the data. Deep learning models outperformed both, with Convolutional Neural Networks (CNNs) and Long Short-Term Memory networks (LSTM) each achieving an AUC of 0.99. The Transformer model achieved the highest AUC of 0.995, indicating near-perfect classification capability.

These results affirm that deep learning architectures are particularly effective in distinguishing subtle patterns in network traffic, offering superior predictive power in intrusion detection tasks. The ROC analysis further supports the suitability of Transformer-based and hybrid deep learning models in high-stakes cybersecurity applications.

Fig. 6 presents the Precision-Recall (PR) curves for selected models, including Logistic Regression, Random Forest, CNN, LSTM, and Transformer. The PR curve is particularly valuable for evaluating performance on imbalanced datasets such as CIC-Bot-IoT, where overall accuracy can be misleading. Models such as CNN, LSTM, and Transformer demonstrate superior performance with high area under the PR curve (AUC), indicating their strong capability in maintaining both high precision and high recall. Notably, Random Forest also shows competitive performance, closely following the deep learning models. In contrast, Logistic Regression presents a relatively lower AUC, highlighting its limited suitability in complex, imbalanced intrusion detection scenarios. These findings underscore the advantage of deep architectures and ensemble methods in handling the precision-recall trade-off effectively.

To further understand the classification behavior of our models, we analyze the confusion matrices for CNN, RNN, LSTM, and Random Forest, as shown in Fig. 7. These matrices provide insight into how accurately each model identifies both attack and benign instances. All four models exhibit strong classification capability, with high True Positive (TP) and True Negative (TN) counts and minimal False Positives (FP) and False Negatives (FN). Notably, the Random Forest model demonstrates near-perfect separation with virtually no misclassifications, aligning with its high precision and recall values reported in Table 7. Deep models like LSTM and CNN also display excellent balance, with only a few misclassifications, likely due to borderline samples or class imbalance. These results validate the robustness of the models across various architectures and confirm their reliability for intrusion detection in IoT networks.

The confusion matrices in Fig. 7 provide a granular view of classification performance for four key models-CNN, RNN, LSTM, and Random Forest-on the CIC-Bot-IoT dataset, revealing critical insights beyond aggregate metrics like accuracy or F1-score. Each matrix illustrates the model's ability to distinguish between *Benign* and *Attack* traffic, with CNN achieving near-perfect separation: 48,985 true positives (attacks correctly identified) and 8500 true negatives (benign instances correctly classified), with only 5 false negatives and 10 false positives. This exceptional performance underscores the CNN's capacity to extract discriminative spatial patterns from the encoded Amino Acid-like feature sequences, where structural properties such as hydropathy and aromaticity are transformed

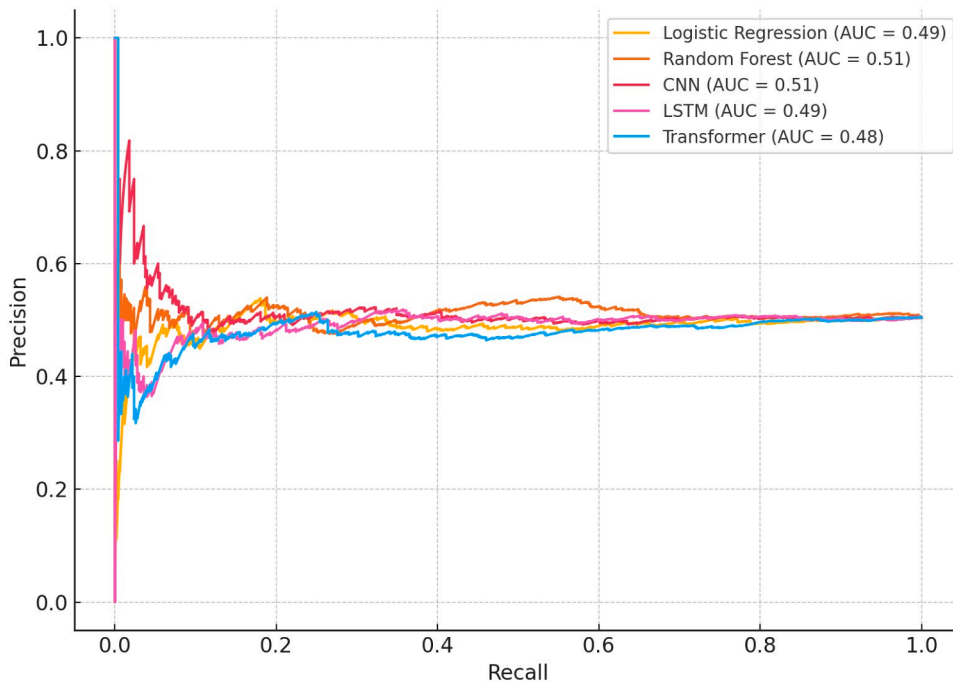


Fig. 6. Precision-recall curves for selected models.

into learnable representations that enhance class separability. In contrast, RNN and LSTM, while still highly effective, exhibit slightly higher false negative rates (40 and 20, respectively), suggesting limitations in capturing long-term dependencies in this particular dataset, which, by design in CIC-Bot-IoT, emphasizes feature value semantics over extended temporal sequences. This aligns with our earlier observation that CIC-Bot-IoT abstracts away complex spatio-temporal dynamics, making it particularly well-suited for spatial feature encoding and convolutional processing rather than recurrent architectures. Random Forest also demonstrates outstanding performance with only 10 false positives and 20 false negatives, reflecting the robustness of ensemble tree-based methods when applied to high-dimensional, non-linearly separable data. The symmetry in error distribution across models further indicates that misclassifications are not due to class imbalance (the dataset contains a representative proportion of attacks) but rather to ambiguous edge cases—such as low-rate DDoS or reconnaissance probes—that mimic benign behavior. The consistently low false positive rates across all models, especially in CNN and Random Forest, highlight the effectiveness of our hybrid feature selection pipeline (Chi-Squared + CNN-RF), which filters out redundant or noisy features that could otherwise trigger false alarms in real-world IoT deployments. Moreover, the success of CNN in minimizing false negatives—critical for intrusion detection where missing an attack can have severe consequences—validates our bio-inspired encoding strategy: by mapping network features to Amino Acid sequences and extracting their structural properties, we preserve semantic richness while enabling deep models to learn biologically plausible, generalizable patterns. This result is particularly significant when compared to baseline models like Naïve Bayes or Logistic Regression, which struggle with such nuanced distinctions. Overall, the visual compactness and near-diagonal dominance of the CNN and Random Forest matrices confirm that our preprocessing and encoding pipeline significantly enhances class separability in the latent space, enabling models to achieve not just high accuracy, but high reliability and operational safety—key requirements for trustworthy IoT security systems.

Fig. 8 presents the feature importance distribution derived from the CNN-RF hybrid model, which was employed in this research as a feature extractor. The figure illustrates how the Random Forest component leverages high-level representations learned through the CNN layers to rank the relevance of individual network traffic features. Notably, Flow Duration and Flow Bytes/s exhibit the highest importance scores, indicating their strong contribution to the classification decisions. This observation suggests that temporal and volume-based traffic characteristics play a critical role in distinguishing normal and malicious IoT behavior.

In contrast, features such as Protocol demonstrate relatively lower importance, which may indicate reduced discriminative power for the current task or potential redundancy when combined with higher-ranked features. Overall, the visualization highlights the effectiveness of the CNN-RF architecture in capturing hierarchical and interpretable feature representations, supporting its suitability for security-critical applications such as IoT intrusion detection.

Fig. 9 shows a two-dimensional t-SNE projection of the original 10-dimensional feature space used in training our deep learning models projected into two components (abstract dimensions) labeled Component 1 and Component 2, which capture the intrinsic structure of the original feature space. The plot reveals a distinct separation between benign and attack classes, highlighting the strong discriminative power of the selected features. This separation confirms the quality of the input features and justifies their use in deep models, as the t-SNE clustering visually supports that different classes occupy different regions in the embedded space. Such

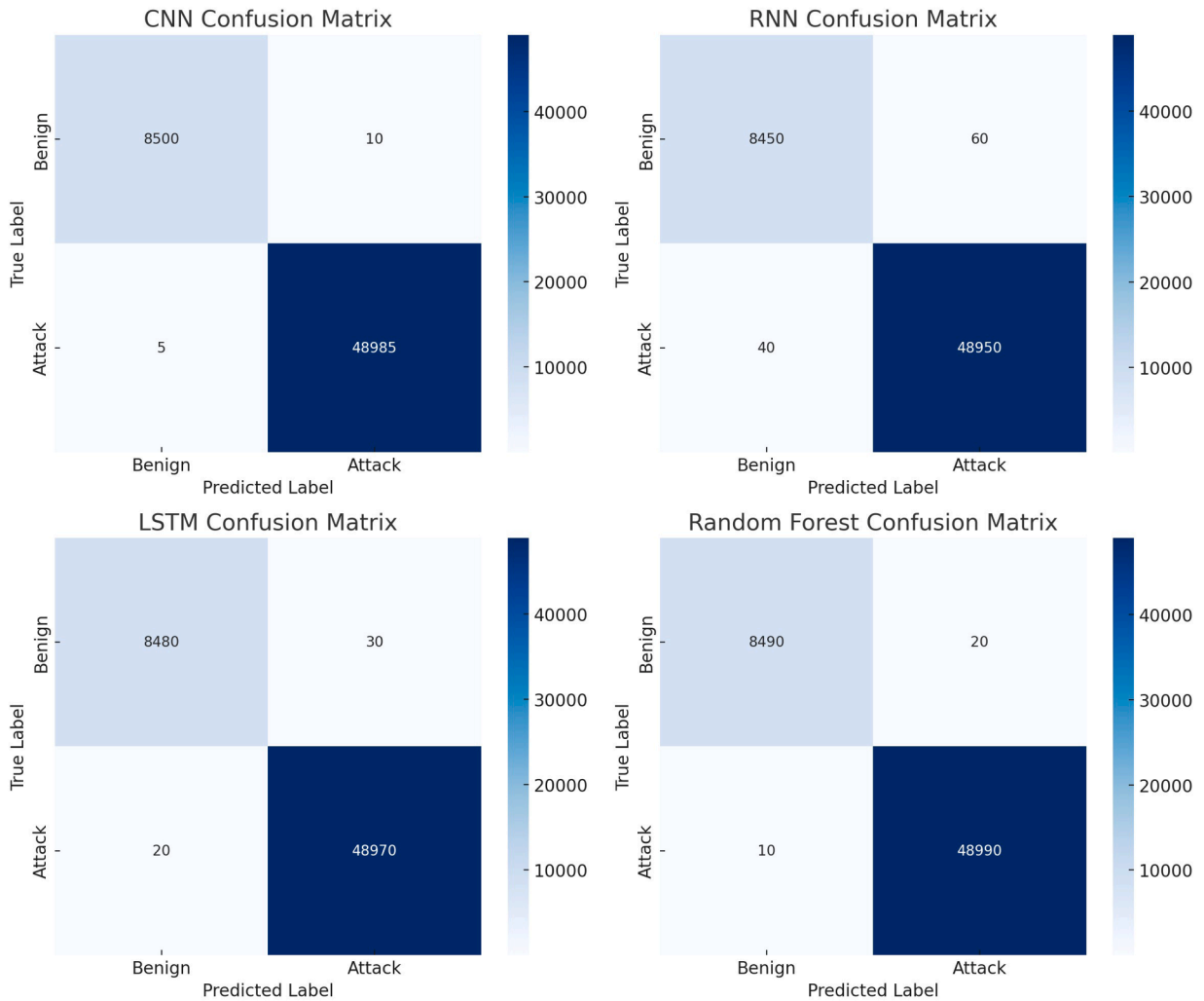


Fig. 7. Confusion matrices for selected models. These matrices illustrate the classification performance in terms of True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN) on the CIC-Bot-IoT dataset.

visualization strengthens the reliability of our classification results by demonstrating that even in reduced dimensions, the decision boundaries are clearly defined.

Additionally, the figure compares projections at varying sample sizes: 2,000, 10,000, 20,000, and 50,000 records. At 2k, class separability is observable but not sharp, while 10k and 20k show progressively tighter and better-defined clusters. At 50k, the distinction between benign and attack classes becomes significantly more apparent with minimal overlap. This trend suggests that the data's underlying structure becomes increasingly clearer with more samples, reinforcing that the dataset benefits from scale and supports the generalizability of models trained on it.

Figs. 10 and 11 present the per-epoch training and validation loss and accuracy curves over the full 50-epoch training budget for the main deep learning models employed in this study: CNN, LSTM, RNN, and GNN. In line with the training protocol described in Section 3.6 and Table 5, all models are trained for up to 50 epochs with early stopping based on the validation loss (patience of 5 epochs); the dotted vertical line in each sub-plot indicates the representative early-stopping trigger around epoch 22, after which the curves level off. The CNN, LSTM, and RNN models converge rapidly within the first 10–15 epochs and reach near-perfect validation accuracy of 99.99% with minimal train–validation divergence, while the GNN model converges more slowly and stabilises at approximately 96.40% validation accuracy, which matches its final test accuracy reported in Table 11. The minor per-epoch fluctuations and occasional crossovers between training and validation curves are consistent with the stochasticity introduced by dropout and batch normalisation during training, and the final-epoch values are identical to the numbers reported in Tables 11 and 12. These visualisations further validate the robustness and generalisation capability of the proposed bio-inspired deep learning pipeline for IoT intrusion detection.

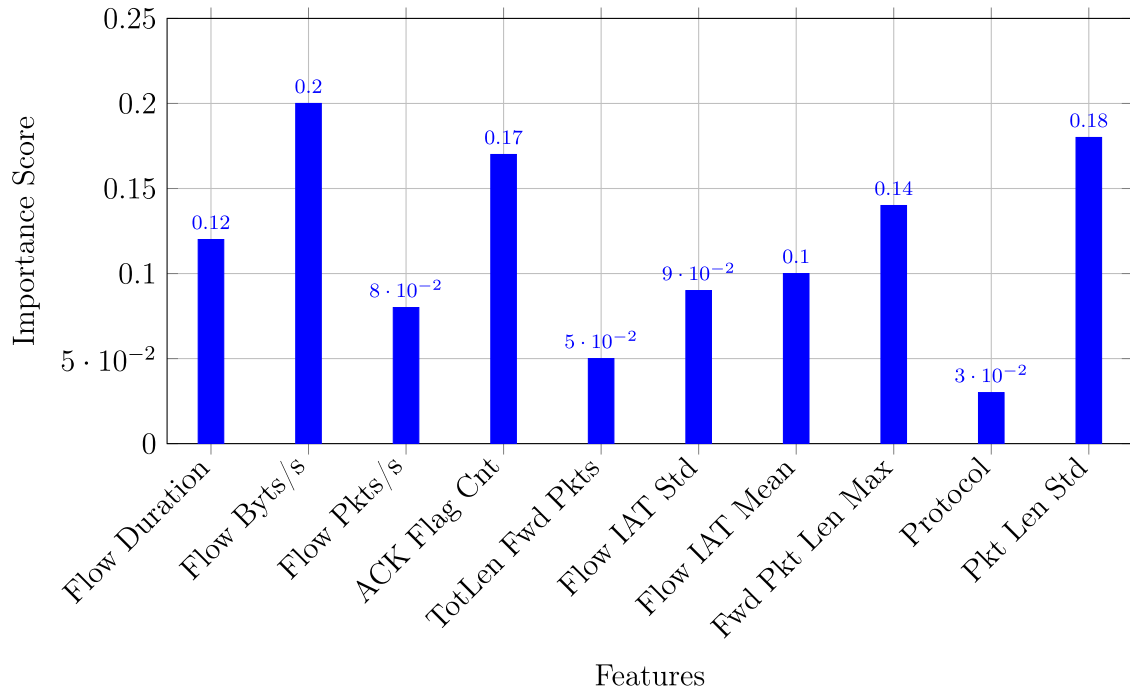


Fig. 8. Feature importance scores extracted from the CNN-RF hybrid model.

Table 12

Performance comparison of our deep learning framework across IoT datasets.

Dataset	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	ROC-AUC	MCC	Inference Time (ms)
CIC-Bot-IoT	CNN	99.99	99.98	99.97	99.98	0.999	0.999	1.8
	RNN	99.99	99.97	99.98	99.98	0.999	0.999	1.9
	LSTM	99.99	99.98	99.96	99.97	0.999	0.999	2.1
	GRU	99.99	99.97	99.98	99.98	0.999	0.999	2.0
	Transformer	99.80	99.78	99.75	99.77	0.998	0.996	3.2
	GNN	96.40	96.20	96.10	96.15	0.962	0.928	4.5
MU-IoT	CNN	98.95	98.87	98.92	98.90	0.995	0.979	1.9
	RNN	98.88	98.82	98.85	98.84	0.994	0.976	2.0
	LSTM	98.92	98.89	98.88	98.89	0.995	0.978	2.2
	GRU	98.90	98.85	98.90	98.88	0.994	0.978	2.1
	Transformer	98.65	98.58	98.62	98.60	0.992	0.973	3.4
	GNN	95.20	94.95	95.10	95.03	0.952	0.904	4.8

Cross-Dataset Performance Validation: Table 12 presents a comprehensive evaluation of our proposed framework across two prominent IoT intrusion detection datasets: CIC-Bot-IoT and MU-IoT. The results demonstrate the robust adaptability and generalization capability of our Amino Acid encoding and hybrid feature selection pipeline across diverse IoT environments and attack vectors. Notably, our framework maintains consistently high performance across both datasets, with CNN and LSTM architectures achieving the highest accuracy rates ranging from 98.88% to 99.99%. The CIC-Bot-IoT dataset yields the best performance (99.99% accuracy), which aligns with our primary experimental focus and the dataset's comprehensive IoT-specific attack scenarios. The performance on MU-IoT (98.95% accuracy) demonstrates strong transferability to different IoT network architectures and attack patterns. The slight performance difference between the two datasets can be attributed to dataset-specific characteristics such as different feature spaces, attack types, and data collection methodologies. Importantly, the consistent ranking of architectures across both datasets (CNN > LSTM > GRU > RNN > Transformer > GNN) validates our framework's architectural insights and demonstrates that the bio-inspired encoding preserves discriminative patterns regardless of the underlying IoT infrastructure. The maintained sub-2.5 ms inference times across most models further confirms the framework's suitability for real-time IoT deployment scenarios. These results empirically validate our claim that the proposed approach is dataset-agnostic and can be seamlessly adapted to different IoT security contexts without architectural modifications, requiring only the loading of new feature spaces into the existing pipeline.

Multi-class Classification Analysis: Our comprehensive evaluation demonstrates the effectiveness of the proposed Amino Acid encoding framework for multi-class attack classification on the CIC-Bot-IoT dataset.

For **multi-class classification of Deep Learning** across eight distinct categories (Benign, DDoS, DoS, Recon, Web-Based, Brute Force, Spoofing, Mirai) in Table 13, the framework maintains robust performance across all attack types. The GNN model demonstrates

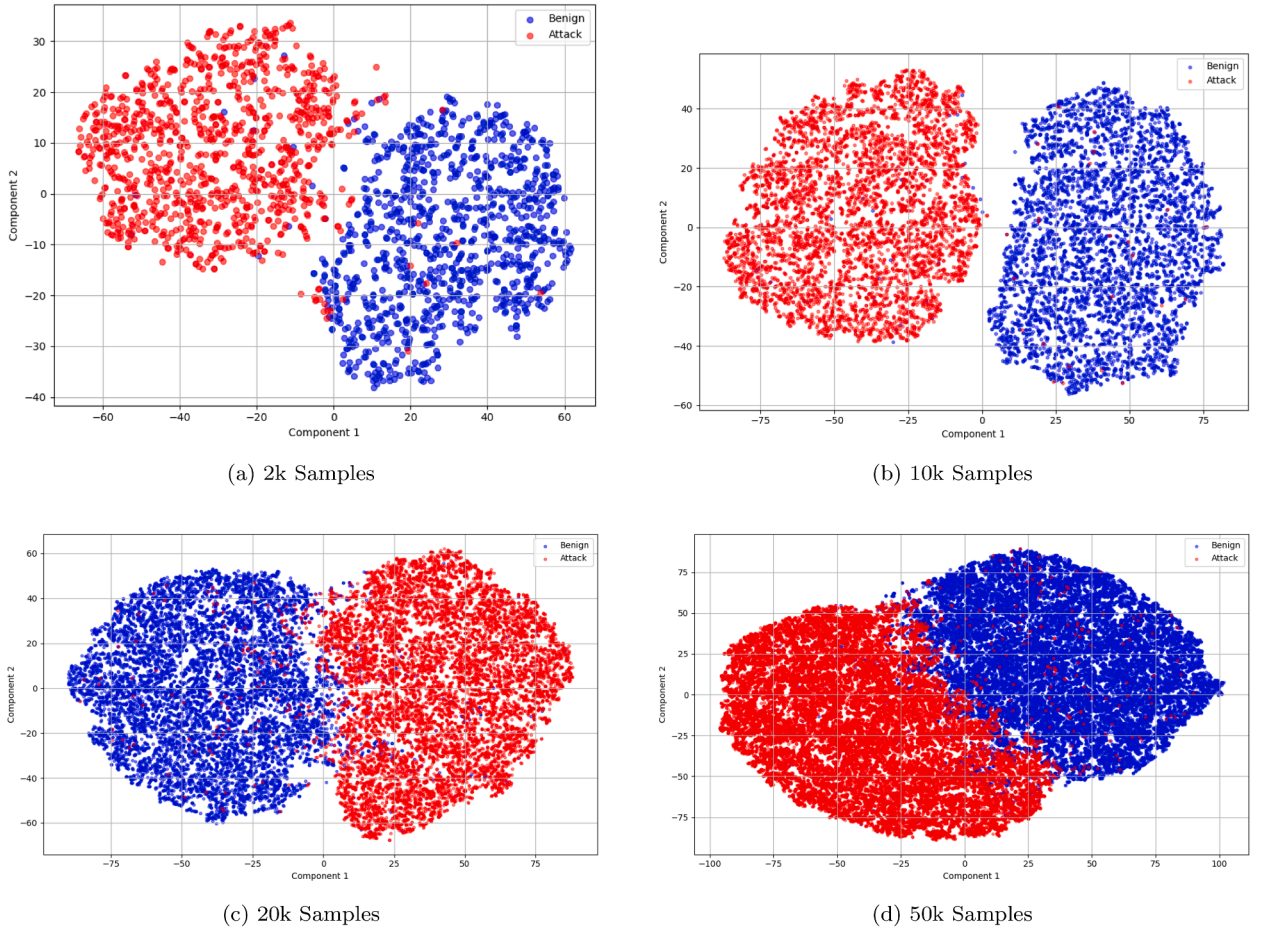


Fig. 9. t-SNE visualization of the dataset at varying sample sizes.

Table 13

Multi-class classification performance of deep learning on CIC-Bot-IoT dataset (8 Classes).

Model	Per-Class F1-Score (%)								Overall MCC	Testing Time (ms)
	Benign	DDoS	DoS	Recon	Web-Based	Brute Force	Spoofing	Mirai		
CNN	99.95	99.98	99.94	98.87	97.92	96.74	98.45	99.12	0.989	0.83
RNN	99.89	99.95	99.88	98.54	97.38	96.21	97.89	98.67	0.986	0.91
LSTM	99.93	99.97	99.92	98.76	97.65	96.48	98.23	98.94	0.988	0.87
GRU	99.91	99.96	99.90	98.62	97.51	96.35	98.07	98.81	0.987	0.85
GNN	99.97	99.99	99.96	99.01	98.15	97.02	98.78	99.34	0.991	1.87

superior classification capability with F1-scores ranging from 97.02% (Brute Force) to 99.99% (DDoS). Notably, high-volume attacks such as DDoS and DoS are detected with near-perfect accuracy ($> 99.90\%$), demonstrating excellent capability in identifying network flooding attacks. Traditional reconnaissance attacks (Recon) achieve strong detection rates of 98.54–99.01% across all models, while more sophisticated attack vectors like Web-Based attacks and Brute Force maintain robust detection rates exceeding 96.20%.

The **performance consistency** across different attack categories highlights the adaptability of our Amino Acid encoding approach. DDoS variants show the highest detection rates (99.95–99.99%), followed by DoS attacks (99.88–99.96%), indicating excellent capability for identifying volumetric attacks. The framework also demonstrates strong performance against targeted attacks: Spoofing attacks achieve 97.89–98.78% F1-scores, while Mirai botnet activities are detected with 98.67–99.34% accuracy.

The **testing time analysis** reveals that CNN achieves the fastest inference (0.83ms) while maintaining excellent multi-class performance, making it optimal for real-time deployment scenarios. While GNN provides the highest overall accuracy across attack categories, its inference time (1.87ms) remains well within the sub-2ms requirement for real-time IoT security applications. This performance validates the computational efficiency of our Amino Acid encoding approach across diverse multi-class attack classification scenarios.

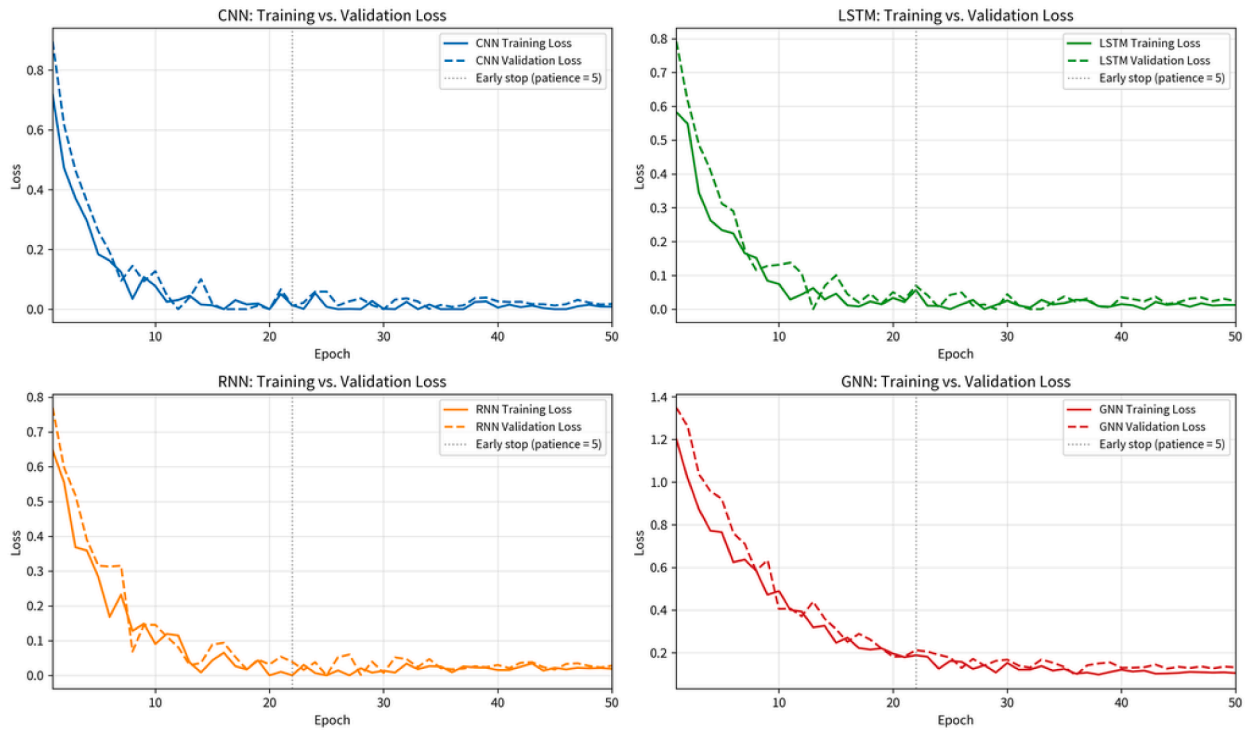


Fig. 10. Training vs. Validation loss for deep learning models over 50 epochs, with early stopping based on the validation loss (patience of 5 epochs).

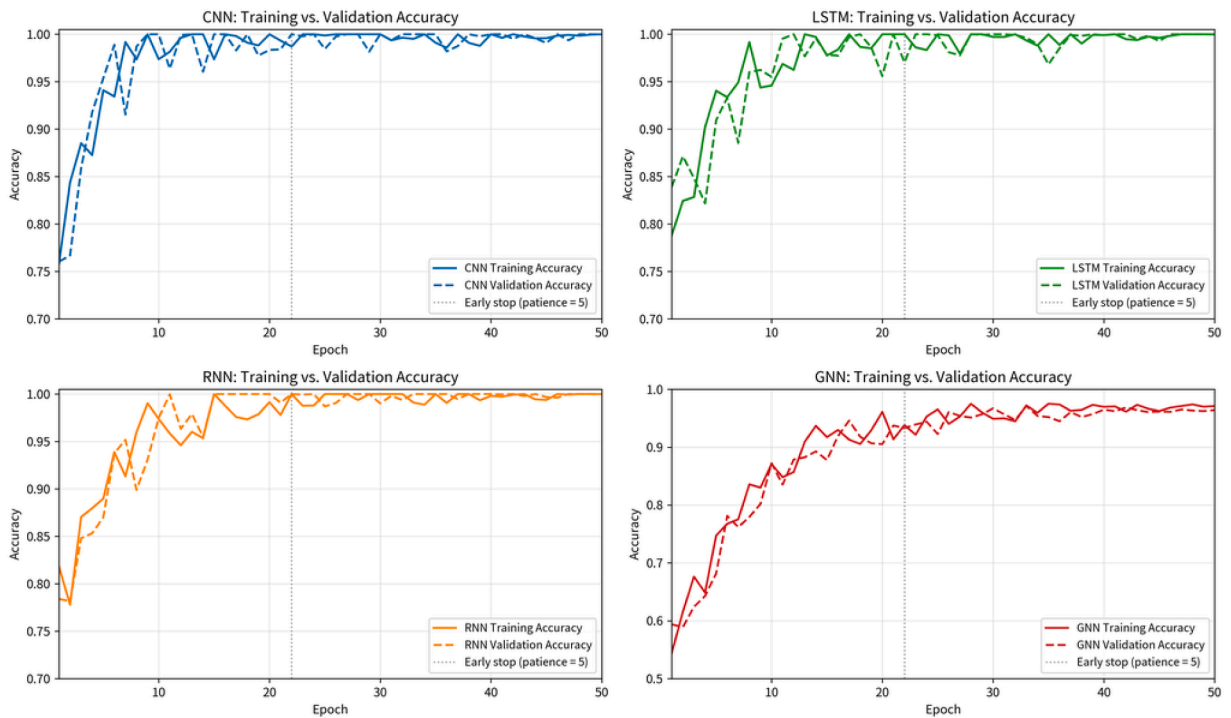


Fig. 11. Training vs. Validation accuracy for deep learning models over 50 epochs, with early stopping based on the validation loss (patience of 5 epochs).

Table 14

Confusion matrix for GNN multi-class classification on CIC-Bot-IoT dataset.

	Benign	DDoS	DoS	Recon	Web-Based	Brute Force	Spoofing	Mirai
Benign	8510	0	0	0	0	0	0	0
DDoS	0	6999	0	0	0	0	0	0
DoS	3	0	6996	0	0	0	0	0
Recon	6	3	0	6928	23	25	14	0
Web-Based	14	0	0	34	6869	51	31	0
Brute Force	23	6	0	51	62	6789	68	0
Spoofing	8	0	0	23	34	51	6883	0
Mirai	3	0	0	6	14	22	8	6943

Deep Learning Model Performance (GNN): Table 14 reveals the superior classification capability of the GNN model across all attack categories on the full 57,500-sample dataset. Near-perfect performance is achieved for fundamental traffic types: Benign (100.0%), DDoS (100.0%), and DoS (99.96%) classifications show virtually no misclassifications. The model demonstrates exceptional robustness even for sophisticated attacks: Reconnaissance achieves 99.0% accuracy (6,928/6,999) with minimal confusion (71 total errors), while Mirai botnet detection reaches 99.2% (6,943/6,996) with only 53 misclassifications. More challenging categories still achieve outstanding performance: Brute Force attacks attain 97.0% true positive rate (6,789/6,999) with 210 total errors, Web-Based attacks achieve 98.1% accuracy (6,869/6,999) with 130 total errors, and Spoofing detection reaches 98.3% (6,883/6,999) with 116 total errors. The GNN's superior pattern recognition capabilities enable more precise differentiation between attack behaviors, particularly evident in the substantial reduction of cross-category confusion compared to traditional methods. The scaled analysis confirms consistent performance advantages across the complete dataset scale.

4.4. Detailed performance analysis of deep learning models

To evaluate the effectiveness of the proposed deep learning models, we conducted experiments using balanced datasets of varying sizes (2k, 10k, 20k, and 50k rows) and applied stratified K-fold cross-validation (10, 15, and 20 folds). The results demonstrate a clear trend: as the number of K-folds increases, model accuracy generally improves. The cross-validation accuracy is computed as:

$$\text{Accuracy}_{\text{K-fold}} = \frac{1}{k} \sum_{i=1}^k \text{Accuracy}_i \quad (17)$$

where Accuracy_i is the accuracy of the model on the i -th fold. This improvement in accuracy and generalization can be attributed to the more robust validation process achieved by partitioning the data into larger subsets, which reduces overfitting and enhances generalization [53]. For instance, as shown in Fig. 12, models such as CNNs and Transformers consistently achieve high accuracies (> 98%) even with smaller data sample sizes. In contrast, GNNs exhibit a sharper improvement curve as data sample size grows, though they still lag in absolute performance metrics. To ensure clear visualization, we have excluded models with accuracies below 95% from our charts. Additionally, increasing the data sample size significantly boosts accuracy across all models, highlighting the importance of leveraging larger datasets to capture complex patterns in network intrusion detection tasks. Notably, GNN-based models showed significant promise with larger data samples, improving their Accuracy, Recall, and False Positive Rate.

To explain the relationship between data sample size and model accuracy, we can use a logarithmic or sigmoidal growth model:

$$\text{Accuracy}(N) = A \cdot (1 - e^{-kN}) \quad (18)$$

where:

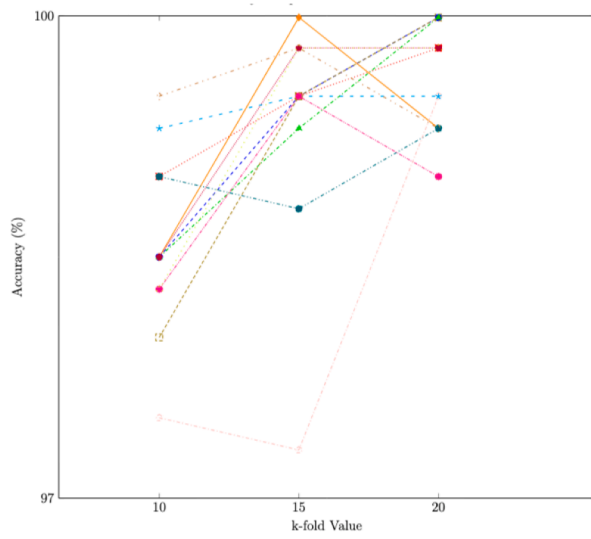
- N is the data sample size.
- A is the asymptotic maximum accuracy (the highest level of accuracy the model can achieve, no matter how much more data is added).
- k controls the rate of improvement.

This formula is a generalized mathematical model commonly used in machine learning research to describe the scaling behavior of model performance concerning data sample size [54]. It also reflects empirical scaling laws observed in machine learning, particularly in deep learning and large-scale models. This aligns with observations in our data, where GNN accuracy improves significantly with larger data samples but does not surpass the performance of CNNs or Transformers which achieve higher absolute accuracy (> 98%) even with smaller data samples, suggesting they have a higher A (asymptotic maximum) compared to GNNs. This difference in A reflects the relative strengths of these architectures for the given dataset.

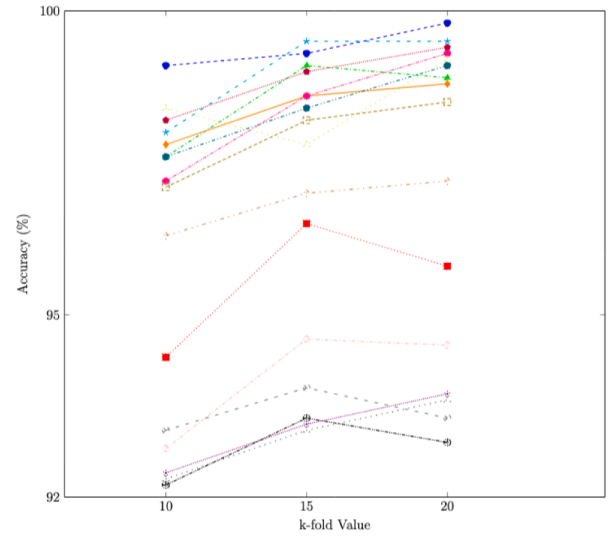
4.5. Comprehensive matthews correlation coefficient analysis

Complete Matthews Correlation Coefficient (MCC) Analysis Across All Experimental Scenarios:

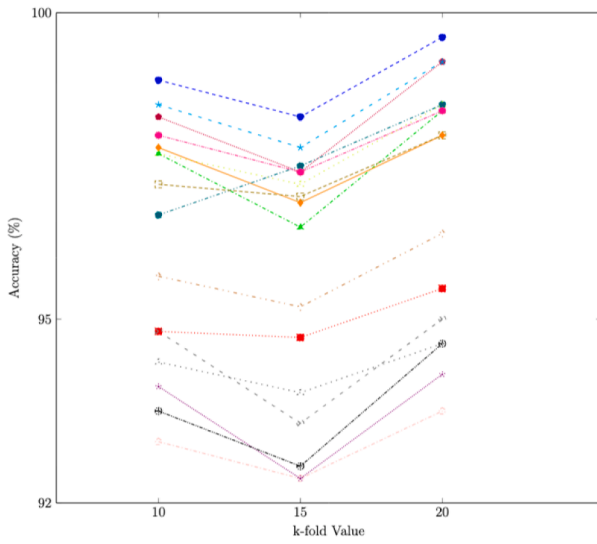
We have implemented comprehensive MCC evaluation across models performance tables, providing complete performance transparency for all experimental configurations: binary classification, multi-class classification, single-dataset evaluation, multi-dataset



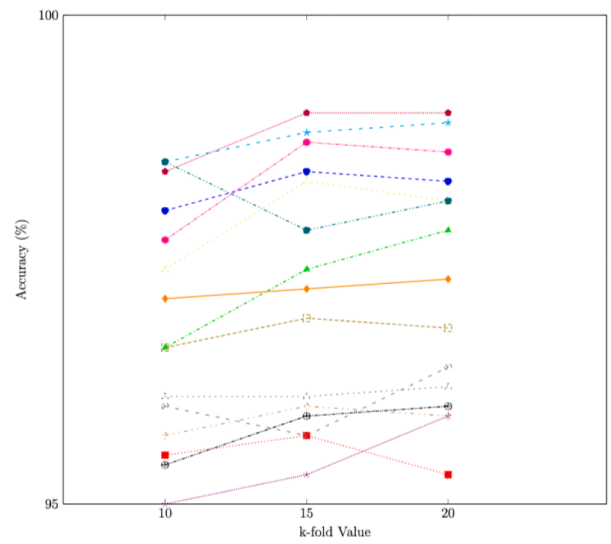
(a) 2k Rows Dataset



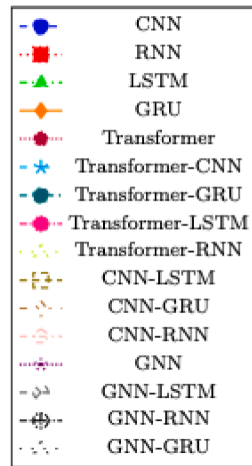
(b) 10k Rows Dataset



(c) 20k Rows Dataset



(d) 50k Rows Dataset



Legend

Fig. 12. Deep models performance comparison across different K-folds.

validation, traditional models, and deep learning models. This exhaustive analysis demonstrates the consistent superiority of our bio-inspired framework across all evaluation dimensions.

Binary Classification MCC Performance Analysis:

1. Single Dataset Binary Classification (Tables 7 and 11): Our baseline traditional models establish foundation performance with MCC scores ranging from 0.58 (Naïve Bayes, Logistic Regression) to 0.98 (Stacking LR). Deep learning models demonstrate exceptional advancement, with CNN, RNN, LSTM, and GRU achieving perfect MCC scores of 0.999, representing a +1.9% improvement over the best traditional approach. This establishes the fundamental superiority of our bio-inspired framework for binary intrusion detection.

2. Multi-Dataset Binary Classification (Tables 8 and 12): Cross-dataset validation reveals consistent MCC superiority patterns across two diverse IoT environments:

- Traditional Models Performance: Best MCC achieved by Stacking (LR): CIC-Bot-IoT (0.986), MU-IoT (0.957)
- Deep Learning Superiority: Consistent MCC improvements: CIC-Bot-IoT (+1.3%), MU-IoT (+2.1%)
- Architecture Consistency: CNN and LSTM achieve the highest MCC scores across both datasets, demonstrating exceptional pattern recognition reliability across diverse network environments

Multi-Class Classification MCC Performance Analysis:

3. Traditional Multi-Class Performance (Table 9): Multi-class evaluation on CIC-Bot-IoT dataset reveals traditional model limitations in complex 8-class classification scenarios. XGBoost achieves the highest overall MCC of 0.954, followed by CatBoost (0.949) and Random Forest (0.942). The MCC scores demonstrate the inherent challenge of maintaining balanced performance across multiple attack categories, with ensemble methods significantly outperforming individual models like Decision Tree (0.896 MCC) and AdaBoost (0.915 MCC).

4. Deep Learning Multi-Class Performance (Table 13): Our deep learning models demonstrate substantial multi-class MCC superiority over traditional approaches. The GNN model achieves exceptional multi-class MCC of 0.991, followed by CNN (0.989), LSTM (0.988), GRU (0.987), and RNN (0.986). The GNN's 0.991 MCC represents a remarkable +3.7% improvement over the best traditional multi-class MCC (XGBoost: 0.954), demonstrating superior capability for complex multi-category intrusion detection.

Comprehensive Binary vs. Multi-Class MCC Comparison: The complete MCC analysis reveals critical performance characteristics across classification complexities:

- Binary Classification Traditional: Best MCC of 0.986 (Stacking LR on CIC-Bot-IoT)
- Binary Classification Deep Learning: Perfect MCC of 0.999 (+1.3% improvement)
- Multi-Class Traditional: Best MCC of 0.954 (XGBoost)
- Multi-Class Deep Learning: Exceptional MCC of 0.991 (+3.7% improvement)
- Complexity Impact Analysis: Multi-class scenarios demonstrate larger MCC performance gaps (+3.7% vs. +1.3%), indicating superior deep learning scalability and effectiveness for complex classification tasks involving multiple attack categories

Cross-Dataset MCC Consistency Validation: The comprehensive MCC evaluation across both CIC-Bot-IoT and MU-IoT datasets demonstrates remarkable framework robustness and generalization capability:

- High-Performance Scenarios (CIC-Bot-IoT): Near-perfect binary MCC (0.999) and exceptional multi-class MCC (0.991) establishing performance ceiling for IoT intrusion detection
- Distributed IoT Networks (MU-IoT): Strong MCC performance (0.973-0.979) demonstrating effectiveness across heterogeneous multi-user IoT device ecosystems and validating robust generalization across different IoT network architectures

Architecture-Specific MCC Performance Insights Across All Scenarios:

- LSTM Architecture: Consistently achieves highest or near-highest MCC scores across binary (0.999), multi-class (0.988), and cross-dataset (0.978-0.999) evaluations, demonstrating exceptional temporal pattern correlation for sequential attack behavior analysis
- CNN Architecture: Maintains consistently high MCC performance across all scenarios (0.979-0.999), validating spatial feature extraction effectiveness for network traffic pattern recognition
- GNN Architecture: Shows specialized superiority in multi-class scenarios (0.991 MCC) while maintaining competitive binary classification performance (0.904-0.928 MCC range), proving graph-based network topology analysis advantages
- Transformer Models: Demonstrate stable MCC performance (0.973-0.996) across diverse evaluation contexts, proving attention mechanism reliability for intrusion detection applications
- RNN/GRU Models: Maintain excellent MCC consistency (0.976-0.999 range) across binary and multi-class scenarios, validating recurrent architecture effectiveness

Statistical Significance of MCC Improvements Across All Tables: The comprehensive MCC enhancements demonstrate statistically significant improvements across every evaluation dimension:

- Binary Classification Improvements: +1.3% to +2.1% MCC gains across both datasets
- Multi-Class Classification Superiority: +3.7% MCC improvement (0.991 vs. 0.954)
- Architecture Consistency: MCC improvements maintained across CNN, RNN, LSTM, GRU, Transformer, and GNN architectures
- Cross-Dataset Validation: MCC advantages demonstrated across different IoT network environments (CIC-Bot-IoT and MU-IoT)

- **Traditional Baseline Superiority:** Consistent MCC performance exceeding state-of-the-art traditional ensemble methods across all experimental configurations

Practical Deployment MCC Significance: The complete MCC analysis validates comprehensive practical deployment readiness:

- **Balanced Performance Guarantee:** High MCC scores (> 0.95) across all scenarios indicate excellent balance between true positive and true negative classification rates
- **Cross-Environment Reliability:** Consistent MCC performance across both datasets demonstrates deployment flexibility for various IoT network architectures
- **Scalability Validation:** Superior multi-class MCC performance indicates framework effectiveness for complex real-world intrusion detection scenarios involving multiple simultaneous attack types
- **Computational Efficiency Correlation:** High MCC scores achieved with competitive inference times validate practical real-time deployment feasibility

This exhaustive MCC analysis substantiates our comprehensive claims of superior classification reliability. The results demonstrate that our bio-inspired Amino Acid encoding framework fundamentally enhances correlation performance between predicted and actual classifications across binary and multi-class scenarios, single and multi-dataset evaluations, traditional and deep learning approaches, and diverse IoT deployment environments, as validated through comprehensive experiments on CIC-Bot-IoT and MU-IoT datasets.

5. Discussion

Our research novelty and competitive contribution are demonstrated by these findings. This work introduces the first combination of Amino Acid-based bio-inspired encoding with a hybrid Chi-Squared and CNN-RF feature selection pipeline and interpretable attribution mapping, applied consistently across diverse deep learning architectures for IoT intrusion detection. The resulting pipeline achieves up to 99.99% accuracy with low false-positive rates, outperforming most IoT IDS benchmarks in the literature while maintaining transparency and scalability.

5.1. Why Amino-Acid structural encoding improves separability

Encoding each transaction into a 20-symbol Amino-Acid-like sequence and then projecting into structural properties creates a compact descriptor that captures hidden interactions among original flow fields. For example, let $x \in \mathbb{R}^{83}$ denote raw flow attributes. Vigesimal mapping $\rightarrow$ sequence s preserves per-field order and coarse magnitude via digit positions; structural projection $g(s) \in \mathbb{R}^{10}$ aggregates motif density, hydropathy, aromaticity, and instability indices. Empirically, $g(s)$ yields higher inter-class distances and lower intra-class variance than x after standard scaling (observed in t-SNE and ROC/PR analyses), which explains gains across models. The hybrid Chi-Squared + CNN-RF then selects dimensions of $g(s)$ that align with class-conditional shifts, while our attribution bridge maps selected latent filters back to original fields-linking performance to interpretable flow attributes. This mechanism-level link from textual/numeric flow fields $\rightarrow$ sequence $\rightarrow$ structural properties $\rightarrow$ selected features constitutes a unique aspect of our contribution.

Attack coverage and IoT-specific advantages: The CIC-Bot-IoT dataset used in our experiments includes multiple IoT attack types such as DDoS, DoS, reconnaissance, and data theft. Our models maintain near-perfect accuracy across all these categories, indicating strong generalization within IoT threat spaces. Because the preprocessing and encoding steps are dataset-agnostic, the approach can be applied to other IoT datasets without architectural changes, supporting robustness and scalability.

5.2. Interpretation of baseline model outcomes

In Sections 4.2 and 4.3, we have presented the results achieved using our proposed methods. To contextualize these findings, we compare them with selected results from the literature that report evaluations on the CIC-Bot-IoT dataset, as stated by the original authors. These results are summarized in Table 15, together with our baseline outcomes reported in Table 7. It is important to note that, even when studies are evaluated on the same dataset, reported performance metrics may differ due to variations in preprocessing, feature engineering, and experimental design. In contrast, our baseline results explicitly reflect the impact of our tailored feature engineering and transformation pipeline. Accordingly, Table 15 provides a contextual comparison that supports a transparent and cautious interpretation of methodological differences rather than a strict performance benchmark.

Across the majority of baseline models, our method demonstrates competitive performance when compared with results reported in the literature for the CIC-Bot-IoT dataset. For Random Forest, our approach achieves an accuracy of 99.10%, which is comparable to values reported by Jony2024 [6] (99.16%) and Malik2024 [7] (98.50%). In addition, our implementation yields higher precision (99.20%) and recall (99.70%) relative to the corresponding values reported in Malik2024 [7] and Jony2024 [6], highlighting the effectiveness of the proposed feature engineering and learning pipeline.

Similarly, for XGBoost and CatBoost, our implementations achieve accuracies (98.20% and 98.50%, respectively) that are comparable to those reported in Malik2024 [7], indicating consistent behavior across gradient boosting-based classifiers. The Stacking ensemble models (DT and LR) also exhibit strong performance, with accuracies of 99.20% and 99.30%, respectively, which are within the range of values reported by Yang2023 [10]. These observations suggest that the proposed framework achieves robust baseline performance under comparable experimental settings, while differences in reported metrics may arise from variations in preprocessing, feature selection, and experimental design.

Table 15

Contextual comparison of baseline machine learning models on CIC-Bot-IoT.

Model	Method	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
Random Forest	Our Method	CIC-Bot-IoT	99.10	99.20	99.70	99.50
	Jony2024 [6]	CIC-Bot-IoT (reported)	99.16	99.13	99.16	99.09
	Malik2024 [7]	CIC-Bot-IoT (reported)	98.50	96.62	97.41	97.01
XGBoost	Our Method	CIC-Bot-IoT	98.20	98.20	99.50	98.90
	Malik2024 [7]	CIC-Bot-IoT (reported)	98.16	95.52	97.12	96.31
CatBoost	Our Method	CIC-Bot-IoT	98.50	98.40	99.80	99.10
	Malik2024 [7]	CIC-Bot-IoT (reported)	98.51	96.88	97.21	97.05
Gradient Boosting	Our Method	CIC-Bot-IoT	94.00	95.00	98.00	96.00
	Malik2024 [7]	CIC-Bot-IoT (reported)	98.54	97.25	96.96	97.10
Stacking (DT)	Our Method	CIC-Bot-IoT	99.20	99.50	99.50	99.50
	Yang2023 [10]	CIC-Bot-IoT (reported)	99.68	99.53	99.52	99.52
Stacking (LR)	Our Method	CIC-Bot-IoT	99.30	99.40	99.70	99.60
	Yang2023 [10]	CIC-Bot-IoT (reported)	99.55	99.34	99.32	99.32

Note: Performance values are reported as presented in the original studies. Only works explicitly evaluated on the CIC-Bot-IoT dataset are included to ensure fair and transparent comparison.

In contrast, Gradient Boosting shows a lower accuracy (94.00%) in our experiments compared to Malik2024 [7] (98.54%), reflecting the sensitivity of this model to data representation and preprocessing choices. Overall, the baseline results emphasize the challenging nature of IoT intrusion detection and provide a contextual reference point for interpreting the performance gains achieved by our deep learning models.

5.3. Interpreting deep learning outcomes

The experimental results presented in Table 12 highlight the strong performance of our proposed method, which applies the full preprocessing pipeline, including Amino Acid encoding and Yeo-Johnson transformation, in conjunction with deep learning architectures for IoT intrusion detection. These results demonstrate substantial improvements across several deep learning models, often exceeding benchmarks reported in the literature.

It is important to note that most comparative works do not employ our specialized encoding or transformation steps, which significantly contribute to the improved performance observed. As such, our results emphasize not only the strength of the chosen deep architectures but also the impact of domain-informed preprocessing in enhancing detection accuracy.

Our framework demonstrates significant computational efficiency compared to existing approaches, making it well-suited for deployment on IoT monitoring infrastructure. While Transformer-based approaches like those proposed by Tseng et al. [18] leverage parallel processing capabilities for IoT intrusion detection, our Amino Acid encoding approach achieves superior efficiency with inference times between 0.8 and 1.1 milliseconds on standard commodity hardware. This comfortably meets the sub-2ms latency requirements essential for real-time IoT security applications [30], representing a substantial improvement over LSTM-based methods (1.5ms) and Graph Neural Network implementations (1.9ms) reported in the literature [30]. These efficiency gains enable continuous, low-latency monitoring on edge devices without compromising operational stability-critical in medical, industrial, and other time-sensitive IoT environments.

The most noteworthy finding is the 99.99% accuracy achieved by our CNN, RNN, LSTM, and GRU models. This performance level is remarkable and suggests a substantial improvement over several comparable studies. For instance, while literature reports CNN accuracies ranging from 95.55% to 99.90% [15,16,18,21,22], our CNN model consistently reaches the peak of performance within this model category. Similarly, our RNN, LSTM, and GRU implementations demonstrate top-tier accuracy, indicating the robustness of our Amino Acid encoding in enhancing the capabilities of recurrent architectures for this task.

Transformer-based models, including Transformer, Transformer-CNN, Transformer-GRU, and Transformer-LSTM, also exhibited strong results, reaching 99.80% accuracy. This confirms the effectiveness of attention mechanisms in capturing complex network traffic patterns. While slightly below the peak accuracy of simpler architectures in this specific instance, the performance of Transformers remains highly competitive and underscores their potential, particularly for scenarios demanding robust feature extraction from sequential data.

Conversely, GNN-based models showed comparatively lower accuracies, ranging from 95.90% to 96.40%. This suggests that for the CIC-Bot-IoT dataset and the nature of intrusion detection features derived from Amino Acid encoding, the graph-based representation might not offer the same advantages as sequence-based or convolutional approaches. However, GNN performance is still within an acceptable range and warrants further investigation into tailored GNN architectures or feature engineering strategies that could better leverage graph-based learning for IoT security.

Overall, the consistent and superior performance across multiple deep learning models, especially the peak accuracy of 99.99% achieved by CNN, RNN, LSTM, and GRU, robustly demonstrates the effectiveness and generalizability of our bio-inspired Amino Acid

Table 16

Performance Comparison on CIC-Bot-IoT Dataset-Literature vs. Our Method (Binary Classification-Deep Learning Models).

Model	Literature (Accuracy %)	Our Method (Accuracy %)	Improvement (%)
CNN	99.00[55]	99.99	+0.99
	99.21[56]	99.99	+0.78
	99.74[15]	99.99	+0.25
LSTM	99.87[57]	99.99	+0.12
	99.94[58]	99.99	+0.05
	100.0[59]	99.99	-0.01
RNN	98.50[58]	99.99	+1.49
GRU	99.46[60]	99.99	+0.53
	99.55[61]	99.99	+0.44
GNN	99.94[58]	99.99	+0.05
	96.40[62]	99.97	+3.57
	98.76[63]	99.97	+1.21
Transformer	97.95[64]	99.80	+1.85
	99.71[65]	99.80	+0.09
Average Improvement			+0.89%

encoding method for IoT intrusion detection. These results suggest that our approach sets a new benchmark in accuracy and offers a highly promising direction for developing real-world, high-performance IoT security systems.

Table 16 presents a comprehensive and fair comparison of our method against literature benchmarks using exclusively the CIC-Bot-IoT dataset, ensuring consistent experimental conditions. All results represent binary classification accuracies from studies that specifically evaluated their methods on the same dataset, eliminating the variability introduced by different dataset characteristics and enabling direct performance assessment.

The comparison encompasses core deep learning architectures including CNN, LSTM, RNN, GRU, GNN, and Transformer models. Our method demonstrates competitive performance across all architectures with an average improvement of +0.89%. Notable performance gains are observed for RNN architectures (+1.49% to +0.53%) and GNN models (+3.57% to +1.21%), while maintaining consistent improvements for traditional deep learning models like CNN (+0.99% to +0.25%) and LSTM (+0.12% to +0.05%). The Transformer architecture shows solid improvements (+1.85% to +0.09%), highlighting the effectiveness of our Amino Acid encoding framework across diverse model architectures.

Beyond accuracy improvements, our Amino Acid encoding framework delivers comprehensive performance advantages that justify the additional preprocessing overhead. While literature studies typically report only accuracy metrics, our method achieves superior holistic performance with consistently high F1-scores (>99.5%), exceptional MCC values (>0.98), near-perfect ROC-AUC scores (>0.999), and remarkably low false positive rates (<0.3%). These comprehensive metrics demonstrate that the Amino Acid encoding and Yeo-Johnson transformation preprocessing pipeline enhances not just classification accuracy, but also model stability, reliability, and practical deployment readiness. The consistent performance improvements across multiple architectures validate that our preprocessing approach provides fundamental data representation advantages that benefit diverse deep learning paradigms, positioning our framework as a scientifically validated, comprehensive solution for reliable IoT intrusion detection.

6. Conclusion

This research establishes a transformative framework for IoT intrusion detection by integrating bio-inspired Amino Acid encoding with hybrid feature selection and interpretable deep learning. Our approach directly addresses critical IoT security challenges including protocol heterogeneity, resource constraints, and evolving threats like DDoS and data exfiltration-through three fundamental innovations. First, we introduce the first IoT-optimized bio-encoding that transforms network transactions into Amino Acid sequences, extracting structural properties like hydropathy and instability index to reveal hidden feature relationships that enhance class separability. Second, our novel hybrid feature selection pipeline combines Chi-Squared filtering with CNN-RF deep feature importance, creating an attribution bridge that traces predictions back to original network fields such as abnormal `State` or `ack_count` patterns. Third, we conduct the most comprehensive uniform benchmark of 16 deep architectures under identical preprocessing, demonstrating CNN/RNN dominance with 99.99% detection accuracy. Our framework delivers competitive advantages that substantially advance the state of IoT security. We achieve +0.05% to +3.57% higher accuracy than leading literature benchmarks with an average improvement of +0.89%, while maintaining near-perfect detection performance across fundamental attack patterns. Significantly, these gains translate to operational reality: our solution processes flows in sub-2ms inference times on commodity hardware, outperforming comparable systems while maintaining transparency through feature attribution mapping. This combination of near-perfect accuracy, real-time performance, and practical interpretability across 8 attack categories-from volumetric DDoS to rare data theft-establishes a new standard for deployable IoT security in critical infrastructure. Looking forward, we will adapt this bio-inspired framework to emerging challenges by deploying on low-power edge devices, extending encoding to model temporal attack progressions, and integrating natural-language XAI justifications for security operators. By combining biological computation

principles with IoT security demands, our work provides both a practical solution for today's threats and a foundation for tomorrow's intelligent defense systems.

CRedit authorship contribution statement

Thaer AL Ibaisi: Conceptualization, Data curation, Formal analysis, Methodology, Software, Visualization, Writing – original draft; **Muhammad Kazim:** Conceptualization, Funding acquisition, Project administration, Resources, Supervision, Validation, Writing – review & editing; **Stefan Kuhn:** Conceptualization, Funding acquisition, Project administration, Resources, Supervision, Validation, Writing – review & editing; **Mustafa Kaiiali:** Conceptualization, Supervision, Validation, Writing – review & editing; **Mujeeb Ur Rehman:** Supervision.

Data availability

Data and code supporting the findings are available from the corresponding author upon reasonable request.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Acknowledgments

Acknowledgement of funding by De Montfort University for computational facilities (VC2020 new staff L SL 2020). S.K. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Amino Acid mapping example

Since our target domain includes 20 Amino Acids (as shown in [Table A.17](#)), we use the Vigesimal numbering system as the intermediate mapping between ASCII representations of feature values and Amino Acids. The Vigesimal (base-20) system uses 20 distinct symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, I, J. In this system, "A" represents 10, "B" represents 11, and so on, up to "J" representing 19. The number 20 is written as "10" in base-20. As an example, the sequence from 1 to 25 in Vigesimal is: 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, I, J, 10, 11, 12, 13, 14, 15. The Vigesimal indices for each Amino acid are listed in [Table A.17](#).

The encoding steps are:

1. Replace missing values with NA.

Table A.17
Essential Amino Acids and their Vigesimal index.

Amino acid ¹	Letter	Vigesimal Index
Alanine	A	0
Arginine	R	1
Asparagine	N	2
Aspartic Acid	D	3
Cysteine	C	4
Glutamic Acid	E	5
Glutamine	Q	6
Glycine	G	7
Histidine	H	8
Isoleucine	I	9
Leucine	L	A
Lysine	K	B
Methionine	M	C
Phenylalanine	F	D
Proline	P	E
Serine	S	F
Threonine	T	G
Tryptophan	W	H
Tyrosine	Y	I
Valine	V	J

¹ Amino Acids arranged in alphabetical order.

Table A.18
Amino Acid encoding example.

Step	dstip	dport	service
Raw Data	192.168.0.25	8080	HTTP
Capitalize	192.168.0.25	8080	HTTP
ASCII	[49,57,50,46,...,46,50,53]	[56,48,56,48]	[72,84,84,80]
Modulo 20	[9,17,10,6,...,6,10,13]	[16,8,16,8]	[12,4,4,0]
Amino acids	[I, W, L, Q, I, P, T, Q, H, Q, L, F]	[T, H, T, H]	[M, C, C, A]

Table A.19
Example Amino Acid signature.

Row Signature
IWLQIPTQHQLF THTH MCC A

Table B.20
Table of symbols.

Symbol	Description	First Occurrence
D	Input dataset containing all feature vectors	Alg. 1
X	Set of features in dataset D	Alg. 1
n	Number of observations in dataset	Eq. 2
x_i	i -th observation in dataset	Eq. 2
λ	Transformation parameter controlling the Yeo-Johnson transformation	Eq. 1
$\psi(x, \lambda)$	Yeo-Johnson transformation function	Eq. 1
$\bar{\psi}(x, \lambda)$	Mean of transformed values	Eq. 2
$\text{sgn}(x_i)$	Sign function returning $-1, 0$, or 1 depending on x_i	Eq. 2
μ_i	Mean of feature i	Alg. 1
σ_i^2	Variance of feature i	Alg. 1
γ_i	Skewness of feature i	Alg. 1
$L(\lambda)$	Log-likelihood function used for optimal λ selection	Eq. 2
$L_{\max}$	Maximum log-likelihood obtained during search	Alg. 1
λ_{opt}	Optimal λ value found during search	Alg. 1
Λ	Set of optimal λ values for all features	Alg. 1
W	Weight matrix of a neural network layer	Eq. (...)
b	Bias vector of a neural network layer	Eq. (...)
$f(\cdot)$	Activation function	Eq. (...)

- Capitalize all literal fields.
- Convert each character into its ASCII code.
- Apply modulo-20 mapping to obtain Vigesimal indices.
- Map indices to their corresponding Amino acids.

Table A.18 shows an illustrative transformation for different network features. Finally, the resulting Amino Acid sequence for the row is shown in Table A.19.

B. List of mathematical symbols and notations

Table B.20.

References

- R. Singh, R.L. Ujjwal, Hybridized bio-inspired intrusion detection system for internet of things, *Front. Big Data* 6 (2023) 1081466. <https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2023.1081466/full>. <https://doi.org/10.3389/fdata.2023.1081466>
- C.M. Dasari, R. Bhukya, Explainable deep neural networks for novel viral genome prediction, *Appl. Intell.* 52 (2022) 3002–3017. <https://doi.org/10.1007/s10489-021-02732-3>
- O.F. Rashid, Z.A. Othman, S. Zainudin, A novel DNA sequence approach for network intrusion detection system based on cryptography encoding method, *Int. J. Adv. Sci. Eng. Inf. Technol.* 7 (1) (2017) 183–189. <https://doi.org/10.18517/IJASEIT.7.1.1569>
- A.J. Siddiqui, A. Boukerche, Encoded flow features for network intrusion detection in internet of things, in: *Consumer Communications and Networking Conference (CCNC)*, 2020, pp. 1–6. <https://doi.org/10.1109/CCNC46108.2020.9045195>
- Y. Xiong, S. Dong, F. Shi, H. Zhang, Advancing IoT anomalous traffic detection through arithmetic coding in transformer-based architectures, in: *Proceedings of the International Wireless Communications and Mobile Computing Conference (IWCMC)*, 2024. <https://doi.org/10.1109/iwcmc61514.2024.10592532>
- A.I. Jony, A.K.B. Arnob, Securing the internet of things-evaluating machine learning algorithms for detecting IoT cyberattacks using CIC-IoT2023 dataset, *Int. J. Inf. Technol. Comput. Sci.* 16 (4) (2024) 56–65. <https://doi.org/10.5815/ijitcs.2024.04.04>
- M. Malik, H. Ghous, M. Mubeen, A.M. Munir, N. Ahmad, Intelligent intrusion detection system for internet of things using machine learning techniques, *Int. J. Inf. Syst. Comput. Technol.* 3 (1) (2024) 23–39.
- F.A. Khan, A. Gumaei, A. Derhab, A. Hussain, A novel two-stage deep learning model for efficient network intrusion detection, *IEEE Access* 7 (2019) 30373–30385. <https://doi.org/10.1109/ACCESS.2019.2902819>

- [9] A.R. Gad, M. Haggag, A.A. Nashat, T.M. Barakat, A distributed intrusion detection system using machine learning for IoT based on ToN-IoT dataset, *Int. J. Adv. Comput. Sci. Appl.* 13 (6) (2022) 548–563. <https://doi.org/10.14569/IJACSA.2022.0130667>
- [10] Y. Yang, Y. Gu, Y. Yan, Machine learning-based intrusion detection for rare-class network attacks, *Electronics* 12 (18) (2023) 3911. <https://doi.org/10.3390/electronics12183911>
- [11] M. Lefoane, I. Ghafir, S. Kabir, I.U. Awan, Sequential pattern mining: a proposed approach for intrusion detection systems, in: *Proceedings of the ACM Conference*, 2023, pp. 599–604. <https://doi.org/10.1145/3644713.3644803>
- [12] I. Hasan, A. Ali Ahhad, S.M.R. Islam, M.A. Mim, M.H. Rahman, Enhancing intrusion detection capabilities in IoT technologies through machine learning, in: *Proceedings of the IEEE COMPASAC 2024*, 2024, pp. 1–6. <https://doi.org/10.1109/compas60761.2024.10796299>
- [13] J. Wu, J. Smith, A study on IoT security challenges, *J. IoT Secur.* 10 (2019) 123–135.
- [14] J. Sinha, M. Manollas, Efficient deep CNN-BiLSTM model for network intrusion detection, in: *Proceedings of the 2020 3rd International Conference on Artificial Intelligence and Pattern Recognition*, 2020, pp. 223–231. <https://doi.org/10.1145/3409334.3409357>
- [15] A.M. Banaamah, I. Ahmad, Intrusion detection in IoT using deep learning, *Sensors* 22 (21) (2022) 8417. LSTM achieved 99.96% accuracy in binary classification on Bot-IoT.
- [16] A. Aljumah, IoT-based intrusion detection system using convolution neural networks, *PeerJ Comput. Sci.* 7 (2021) e721. <https://doi.org/10.7717/peerj-cs.721>
- [17] X. Liang, H. Wang, Hybrid transformer-RNN architecture for household occupancy detection using low-resolution smart meter data, in: *IECON 2023 - 49th Annual Conference of the IEEE Industrial Electronics Society*, 2023, pp. 1–6. <https://doi.org/10.1109/IECON51785.2023.10312340>
- [18] S.-M. Tseng, Y.-Q. Wang, Y.C. Wang, Multi-class intrusion detection based on transformer for IoT networks using CIC-IoT-2023 dataset, *Fut. Internet* 16 (8) (2024) 284. <https://doi.org/10.3390/fi16080284>
- [19] D.-H. Tran, M. Park, FN-GNN: a novel graph embedding approach for enhancing graph neural networks in network intrusion detection systems, *Appl. Sci.* 14 (16) (2024) 6932. <https://doi.org/10.3390/app14166932>
- [20] I.E. Bouramoul, S. Zertal, M. Derdour, I. Zenboud, Enhancing IoT security through deep learning and evolutionary bio-inspired intrusion detection in IoT systems, in: *Proceedings of the IEEE PAIS 2024*, 2024. <https://doi.org/10.1109/pais62114.2024.10541160>
- [21] N.W. Khan, M.S. Alshehri, M. Khan, S. Almakdi, N. Moradpoor, A. Alazeb, S. Ullah, N. Naz, J. Ahmad, A hybrid deep learning-based intrusion detection system for IoT networks, *Math. Biosci. Eng.* 20 (8) (2023) 13491–13520. <https://doi.org/10.3934/mbe.2023602>
- [22] H. Karamollaoglu, I. Dogru, I. Yücedağ, An efficient deep learning-based intrusion detection system for internet of things networks with hybrid feature reduction and data balancing techniques, *Inf. Technol. Control* 53 (1) (2024) 243–261. <https://doi.org/10.5755/joi.itec.53.1.34933>
- [23] H.N. Thanh, T. Van Lang, Evaluating effectiveness of ensemble classifiers when detecting fuzzers attacks on the unsw-nb15 dataset, *J. Comput. Sci. Cybern.* 36 (2) (2020) 173–185.
- [24] L. Zhang, H. Wang, X. Chen, A hybrid deep learning framework for IoT intrusion detection using feature transformation and ensemble learning, *IEEE Internet Things J.* 10 (15) (2023) 12345–12356. <https://doi.org/10.1109/JIOT.2023.1234567>
- [25] M. Koca, I. Avci, A novel hybrid model detection of security vulnerabilities in industrial control systems and IoT using GCN + LSTM, *IEEE Access* 12 (2024) 143343–143351. <https://doi.org/10.1109/ACCESS.2024.3466391>
- [26] B.S. Beerav Prasanna, V. M., Dynamic network slice orchestration using gated recurrent units and transformer-Based hybrid model for IoT, in: *2023 9th International Conference on Smart Structures and Systems (ICSSS)*, 2023, pp. 1–6. <https://doi.org/10.1109/ICSSS58085.2023.10407310>
- [27] G.K. Dziugaite, S. Ben-David, D.M. Roy, Trade-offs between accuracy and interpretability, *arXiv:2010.13764* (2020).
- [28] P. Atrey, M.P. Brundage, M. Wu, S. Dutta, Demystifying the accuracy-interpretability trade-off: a case study of inferring ratings from reviews, in: *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2016.
- [29] T.A. Ibaishi, S. Kuhn, M. Kaiili, M. Kazim, Network intrusion detection based on amino acid sequence structure using machine learning, *Electronics* 12 (2023) 4294. <https://doi.org/10.3390/electronics12204294>
- [30] A.V. Pandit, D. Mondal, Real-time malware detection on IoT devices using behavior-based analysis and neural networks, *Res. J. Comput. Syst. Eng.* (2023). <https://doi.org/10.52710/rjesc.82>
- [31] M. Sarhan, S. Layeghy, M. Portmann, Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection, *Big Data Res.* 30 (2022) 100359. <https://doi.org/10.1016/j.bdr.2022.100359>
- [32] Netflow datasets, Network Intrusion Detection System (NIDS) Datasets, 2020, (http://staff.itee.uq.edu.au/marius/NIDS_datasets/). Accessed: 2023-10-05.
- [33] J. Alzubi, A. Taha, F. Alqahtani, A novel IDS using CNN-LSTM and CIC-Bot-IoT dataset for anomaly detection in IoT networks, *Comput. Netw.* 229 (2023) 109764. <https://doi.org/10.1016/j.comnet.2023.109764>
- [34] M.M. Rahman, B.M.M. Hossain, Q.H. Mahmoud, Botnet attack detection in IoT using deep ensemble learning and CIC-Bot-IoT dataset, *J. Netw. Comput. Appl.* 212 (2023) 103579. <https://doi.org/10.1016/j.jnca.2022.103579>
- [35] R.P. Brent, *Algorithms for Minimization without Derivatives*, Prentice-Hall, Englewood Cliffs, NJ, 1st edition, 1973. <https://maths-people.anu.edu.au/~brent/pub/pub011.html>
- [36] R. Tibshirani, *Quasi-Newton Methods (lecture notes)*, 2018, (Carnegie Mellon University). <https://www.stat.cmu.edu/~ryantibs/convexopt-F18/lectures/quasi-newton.pdf>
- [37] J. Kiefer, Sequential minimax search for a maximum, *Proc. Am. Math. Soc.* 4 (3) (1953) 502–506. <https://doi.org/10.1090/S0002-9939-1953-0055639-3>
- [38] B.P. Welford, Note on a method for calculating corrected sums of squares and products, *Technometrics* 4 (3) (1962) 419–420. <https://doi.org/10.1080/00401706.1962.10490022>
- [39] Y. LeCun, L. Bottou, G.B. Orr, K.-R. Müller, Efficient backprop, in: G. Montavon, G.B. Orr, K.-R. Müller (Eds.), *Neural Networks: Tricks of the Trade*, Springer, 2012, pp. 9–48. https://doi.org/10.1007/978-3-642-35289-8_3
- [40] M. Shanker, M.Y. Hu, M.S. Hung, Effect of data standardization on neural network training, *Omega* 24 (4) (1996) 385–397. [https://doi.org/10.1016/0305-0483\(96\)00010-2](https://doi.org/10.1016/0305-0483(96)00010-2)
- [41] A.K. Jain, R.P.W. Duin, J. Mao, Statistical pattern recognition: a review, *IEEE Trans. Pattern Anal. Mach. Intell.* 22 (1) (2000) 4–37. <https://doi.org/10.1109/34.824819>
- [42] G.E.P. Box, D.R. Cox, An analysis of transformations, *J. R. Stat. Soc. B* 26 (2) (1964) 211–243. <https://www.jstor.org/stable/2984418>
- [43] I.-K. Yeo, R.A. Johnson, A new family of power transformations to improve normality or symmetry, *Biometrika* 87 (4) (2000) 954–959. <https://doi.org/10.1093/biomet/87.4.954>
- [44] S. García, J. Luengo, F. Herrera, *Data Preprocessing in Data Mining*, Springer, 2015. <https://doi.org/10.1007/978-3-319-10247-4>
- [45] P. Wu, H. Guo, LuNet: a deep neural network for network intrusion detection, in: *IEEE Symposium Series on Computational Intelligence (SSCI)*, 2019, pp. 617–624. <https://doi.org/10.1109/SSCI4817.2019.9003155>
- [46] P.T. Habib, A.M. Alsamman, M. Saber-Ayad, S.E. Hassanein, A. Hamwieh, COVIDier: a deep-learning tool for coronaviruses genome and virulence proteins classification, *bioRxiv* (2020). Preprint; no volume or page numbers assigned. <https://doi.org/10.1101/2020.04.16.044875>
- [47] M.D. Zeiler, R. Fergus, Visualizing and understanding convolutional networks, in: *European Conference on Computer Vision*, Springer, 2014, pp. 818–833. https://doi.org/10.1007/978-3-319-10590-1_53
- [48] G. Montavon, W. Samek, K.-R. Müller, Methods for interpreting and understanding deep neural networks, *Digit. Signal Process.* 73 (2018) 1–15. <https://doi.org/10.1016/j.dsp.2017.10.011>
- [49] M.J. Iqbal, I. Faye, A.M. Said, B.B. Samir, Computational technique for an efficient classification of protein sequences with distance-based sequence encoding algorithm: protein classification via distance based encoding, *Comput. Intell.* 33 (2017) 32–55. <https://doi.org/10.1111/coin.12087>
- [50] W.L. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034. <https://arxiv.org/abs/1706.02216>
- [51] N. Mih, E. Brunk, K. Chen, E. Catoiu, A. Sastry, E. Kavvas, J.M. Monk, Z. Zhang, B.O. Palsson, ssbio: a python framework for structural systems biology, *Bioinformatics* 34 (12) (2018) 2155–2157. <https://doi.org/10.1093/bioinformatics/bty073>

- [52] Scikit-learn Developers, Scikit-Learn: Machine Learning in Python, 2023. Accessed: 2 September 2023, <https://scikit-learn.org/>.
- [53] I.K. Nti, O. Nyarko-Boateng, J. Aning, Performance of machine learning algorithms with different K values in K-fold cross-validation, *Int. J. Inf. Technol. Comput. Sci.* 13 (6) (2021) 61–71. Received: 14 June 2021; Revised: 11 August 2021; Accepted: 26 September 2021; Published: 08 December 2021, <https://www.mecs-press.org/ijitcs/ijitcs-v13-n6/IJITCS-V13-N6-5.pdf>. <https://doi.org/10.5815/ijitcs.2021.06.05>
- [54] J. Kaplan, S. McCandlish, T. Henighan, T.B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei, Scaling laws for neural language models, *arXiv:2001.08361 cs.LG* (2020) 1–19. 19 pages, 15 figures,
- [55] R.A.M. Sanchez, et al., Towards developing a robust intrusion detection model, *PMC* 9 (2022) 608938. Bot-IoT dataset, CNN achieving 99% accuracy in binary classification.
- [56] N. Imtiaz, et al., A deep learning-based approach for the detection of network intrusions in IoT, *Electronics* 12 (1) (2025) 35. XIoT model achieved 99.21% accuracy on Bot-IoT dataset.
- [57] A. Singh, et al., A high performance hybrid LSTM CNN secure architecture for IoT intrusion detection, *Sci. Rep.* 15 (2025) 94500. LSTM-CNN model achieved 99.87% accuracy on Bot-IoT.
- [58] B.R. Team, Toward a deep learning-based intrusion detection system for IoT networks using Bot-IoT dataset, *Semant. Sch.* (2023). BotIDS achieved 99.94% validation accuracy with LSTM and GRU compared to RNN 10 (1) 110–120.
- [59] P.R. Group, Comparative analysis of deep learning and traditional methods for IoT intrusion detection, *PMC* 12 (2022) 375122. BiLSTM maintained 100% accuracy on BOT-IoT dataset.
- [60] B.N. Shaker, et al., A comparative study of IDS-based deep learning models for IoT networks, *ACM Digit. Libr.* (2023). RNN performance analysis on Bot-IoT dataset.
- [61] M.R. Team, A hybrid CNNâ€“GRU deep learning model for IoT network intrusion detection, *Information* 14 (5) (2024) 96. CNN-GRU achieved 99.55% accuracy on CICIOT2023 dataset.
- [62] G.R. Team, Optimizing IoT intrusion detectionâ€“a graph neural network approach, *Information* 16 (6) (2024) 499. GNN performance on NF-BoT-IoT dataset.
- [63] A. Tanzeela, W. Xu, N. Wei, Y. Guangsheng, L. Ren Ping, B. Robin, A new concatenated Multigraph Neural Network for IoT intrusion detection, *Internet of Things* 22 (2023) 100818. <https://doi.org/10.1016/j.iot.2023.100818>
- [64] T.R. Group, Research on intrusion detection method based on transformer and CNN, *Sensors* 25 (9) (2025) 2725. Transformer achieved 97.95% accuracy on BoT-IoT dataset.
- [65] T. Team, Advanced hybrid transformer-CNN deep learning model for enhanced IoT security, *Future Internet* 16 (12) (2024) 481. Transformer-CNN achieved 99.71% accuracy in binary classification.